

GOETHE UNIVERSITY FRANKFURT
DEPARTMENT OF COMPUTER SCIENCE AND MATHEMATICS

BACHELOR'S THESIS

Variations of the $n \times 2$ -Puzzle and Appropriate Algorithms

BY

Viktor Rieß

SUPERVISOR

Prof. Dr. Ulrich Meyer

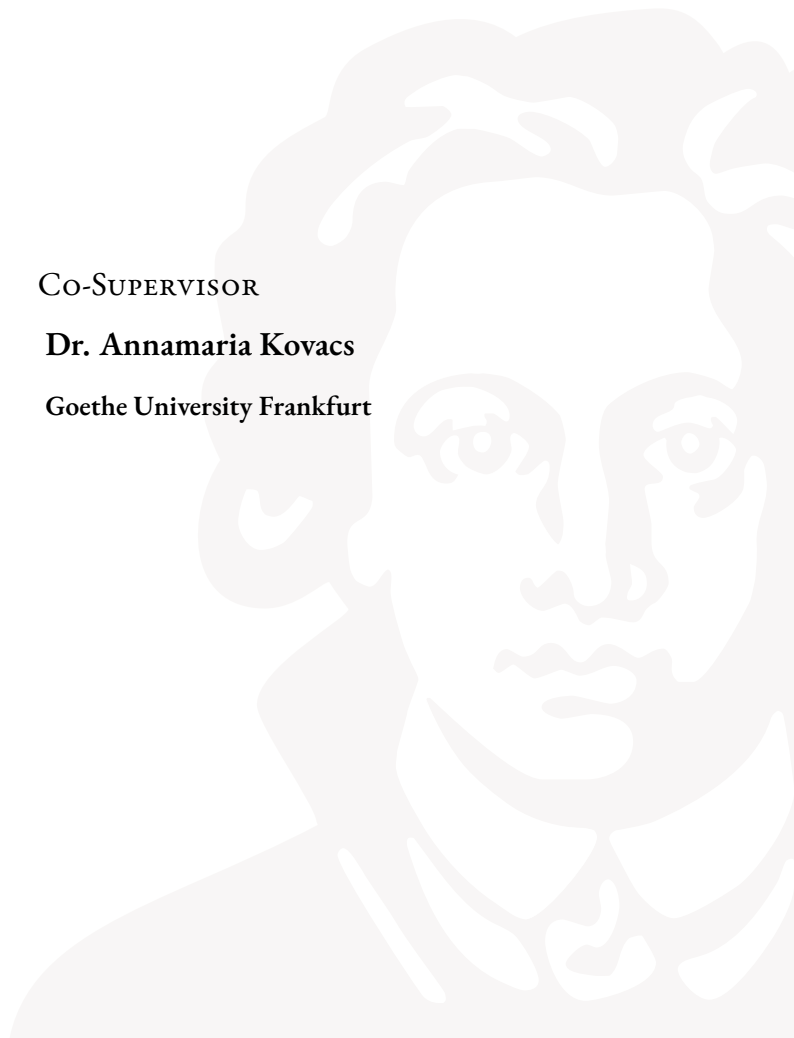
Goethe University Frankfurt

CO-SUPERVISOR

Dr. Annamaria Kovacs

Goethe University Frankfurt

JANUARY 18, 2026



Abstract

We study a sliding puzzle variant defined on an $n \times 2$ -sized grid, where tiles can be moved into an adjacent empty space. Each tile carries a label, and the goal is to sort the puzzle according to these labels.

In this work, three variations of this puzzle are proposed, each varying in complexity and in the number of distinct labels used. We examine the solvability conditions for each variation and introduce two core algorithmic components: a procedure that solves a single row and a routine that separates the puzzle tiles into two groups. These components serve as building blocks for multiple algorithms solving the different puzzle variations.

We study the number of moves required by each algorithm through both theoretical analysis and empirical evaluation. The solutions produced by each algorithm are visualized through a web-based interface. The results of this thesis provide insights into algorithmic design for puzzles of general size.

CONTENTS

1	INTRODUCTION	1
1.1	Previous Work	3
1.2	Our Results	4
2	PRELIMINARIES	5
2.1	Modelling the $n \times 2$ -Puzzle	5
2.1.1	The Puzzle Board State	5
2.1.2	Manipulating the Puzzle	7
2.1.3	The Goal of the Puzzle	8
2.2	Solvability	9
2.2.1	Numbered Variation	9
2.2.1.1	Algorithmically Deciding Solvability in the Numbered Variation	18
2.2.2	Colored Variations	21
3	ALGORITHMS	24
3.1	Complexity of $n \times 2$ -Puzzles	24
3.2	Optimal Algorithm	26
3.3	solveRow-Procedure	29
3.4	partition-Procedure	32
3.5	Algorithms	38
3.5.1	RowByRow	39
3.5.2	Numbered Variation: DIVIDEONCE	40
3.5.3	Numbered Variation: DIVIDEANDCONQUER	41
3.5.4	2-Color Variation: PARTITIONONCE	45
3.5.5	k -Color Variation: COLORBYCOLOR	45
4	EMPIRICAL ANALYSIS	48
4.1	Test Results and Evaluation	48
4.1.1	Numbered Variation	49
4.1.2	2-color Variation	50
4.1.3	k -color Variation	51
4.1.4	General Observations	53

5	CONCLUSION	54
	BIBLIOGRAPHY	56
A	SUPPLEMENTARY MATERIAL	59
B	TOOLS AND AIDS	60

1 INTRODUCTION

The *15-puzzle* is a *sliding puzzle* that was invented in the 1870s in the United States. It rose to popularity quickly — also spreading to Europe in the process — and in 1879, the first scientific article on 15-puzzles was published in *The American Journal of Mathematics* [1]. At the time, it was apparently regarded as too trivial a subject to warrant a standalone article. Therefore, in an appendix to their paper, the authors Johnson and Story provided a justification for its publication, stating:

The “15” puzzle for the last few weeks has been prominently before the American public, and may safely be said to have engaged the attention of nine out of ten persons of both sexes and of all ages and conditions of the community. But this would not have weighed with the editors to induce them to insert articles upon such a subject in the American Journal of Mathematics, but for the fact that the principle of the game has its root in what all mathematicians of the present day are aware constitutes the most subtle and characteristic conception of modern algebra, viz: the law of dichotomy applicable to the separation of the terms of every complete system of permutations into two natural and infeasible groups, a law of the inner world of thought, which may be said to prefigure the polar relation of left and right-handed screws, or of objects in space and their reflexions in a mirror. Accordingly the editors have thought that they would be doing no disservice to their science, but rather promoting its interests by exhibiting this a priori polar law under a concrete form, through the medium of a game which has taken so strong a hold upon the thought of the country that it may almost be said to have risen to the importance of a national institution. [1]

The puzzle’s quick rise to popularity may well be attributed to its simplicity. The game is played on a small square board containing 15 square tiles numbered 1 to 15. The board has 16 cells where the tiles are placed in arbitrarily. This leaves one cell empty; the adjacent tiles can be slid into this empty space. Doing this, the goal is to change the arrangement of the tiles so that they are ordered as depicted in [Figure 1.1](#).

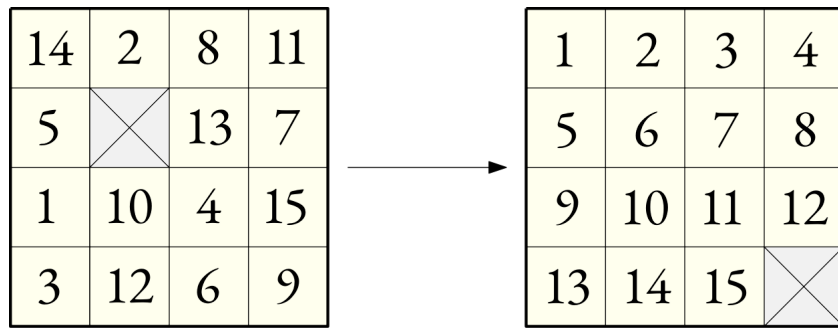


Figure 1.1: To the left, we see the square 15–puzzle board with tiles from 1 to 15 placed in it. The tiles with numbers 5, 2, 10 and 13 can be moved into the grey empty space. By repeatedly moving tiles, we want to rearrange them so that the puzzle board ends up as on the right; the tiles are ordered left to right, top to bottom and the empty space is in the bottom right corner.

It must be noted that it is not always possible to solve the 15–puzzle. If you were to place the tiles anywhere on the puzzle board randomly, there is a 50% chance that the tiles cannot be rearranged to reach the goal no matter how often they are slid around [1].

Nonetheless, years later, the puzzle maker Sam Loyd offered 1000 Dollars (which is equivalent to more than 35,000 Dollars today) to the first person that could solve a 15–puzzle he proposed. This 15-puzzle was in fact impossible to solve; he was probably aware that he would not be losing his money. Loyd claimed decades later that he had driven “the entire world crazy” with his prize. This is to be taken with a grain of salt; Loyd also falsely stated to have invented the 15–puzzle. In Loyd’s book [2], he illustrated the frenzy he had supposedly caused (see Figure 1.2) and wrote:

People became infatuated with the puzzle and ludicrous tales are told of shopkeepers who neglected to open their stores; of a distinguished clergyman who stood under a street lamp all through a wintry night trying to recall the way he had performed the feat. . . . Pilots are said to have wrecked their ships, and engineers rush their trains past stations. A famous Baltimore editor tells how he went for his noon lunch and was discovered by his frantic staff long past midnight pushing little pieces of pie around on a plate! [2]

Today, 15–puzzles have taken on new forms and have been adapted in many ways. For example, the tiles sometimes have pictures or letters on them instead of numbers or the puzzle board has a larger size. Rectangular puzzle boards exist as well. There is also a variety of apps and websites where the game can be played.

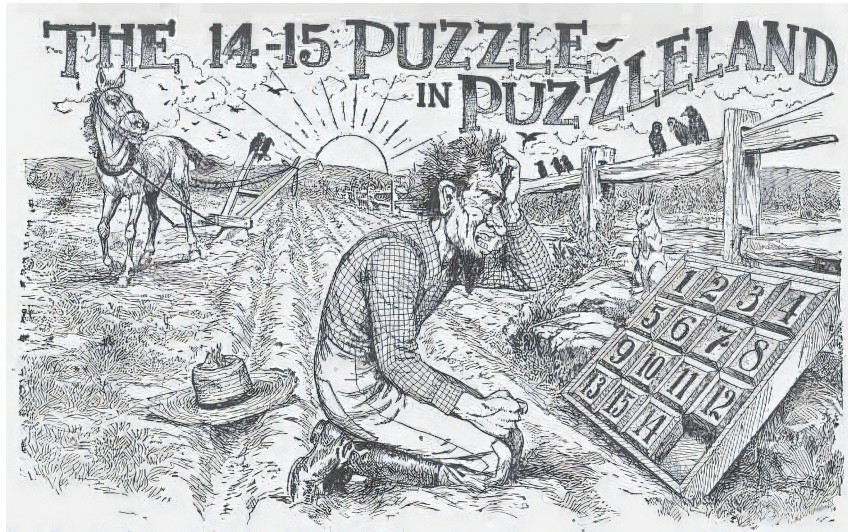


Figure 1.2: Sam Loyd’s illustration of how “farmers have deserted their plows” because of his 1000 Dollar prize. To obtain the money, one had to solve the 15–puzzle depicted on the right. Every tile is placed correctly except for the tiles with numbers 14 and 15. This puzzle is impossible to solve.

1.1 PREVIOUS WORK

In science, the puzzle has been thoroughly studied and generalized since first becoming popular. This subsection presents some of the most important results.

In their work from 1999, Brüngger et al. used parallel search methods to show that every 15–puzzle can be solved in at most 80 moves [3]. In 2005, Korf and Schultze were able to compute an optimal solution (i.e. a solution with the least number of moves) for every possible starting arrangement of the board [4].

One generalization of 15–puzzles is the $(n^2 - 1)$ -puzzle, which has tiles arranged in n rows and n columns as compared to the 15–puzzle with $n = 4$. Ratner and Warmuth showed in 1986 that in $(n^2 - 1)$ -puzzles, finding the optimal solution is $\mathcal{NP}$ -hard. Furthermore, they proved that approximating an optimal solution by an additive constant is $\mathcal{NP}$ -hard as well. Nonetheless, it is possible to design a polynomial time approximation algorithm for finding a solution with the number of moves being larger than the optimal solution by a constant factor [5].

The $\mathcal{NP}$ -hardness of the puzzle combined with its easy implementation make it useful for testing search algorithm heuristics in artificial intelligence. In May 2023, Takano developed a self-supervised Deep Neural Network that can compute near-optimal solutions for 15–puzzles given sufficient resources [6].

Other generalizations of sliding puzzles have been introduced and studied, such as graph sliding puzzles [7] and higher dimensional sliding puzzles [8].

In this article, we are interested in the rectangular $n \times m$ -puzzle with n rows and m columns. More specifically, we examine the $n \times 2$ -puzzle where there are n rows, and only 2 columns. The $n \times 2$ case was studied in 2023 by Zhong, who designed an algorithm with $O(n \log n)$ additional moves compared to an optimal solution [9].

1.2 OUR RESULTS

In this work, we deal with the $n \times 2$ -puzzle. We introduce and study 3 different variations of the puzzle:

- The *numbered* variation. It is directly derived from the 15-puzzle; Each tile on the puzzle board is labeled with a unique integer from 1 to $2n - 1$.
- The *2-color* variation. The tiles are colored either black or white. The goal of the puzzle is to put the black tiles towards the top and the white tiles towards the bottom of the board.
- The *k-color* variation. Each tile has one of k colors, with k being an integer between 3 and $2n - 2$. The colors have a defined order and solving the puzzle means sorting the tiles according to that order.

In [Chapter 2](#), we provide a formal introduction to the puzzle and its variations and examine whether the puzzles can be solved in the different variations or not. [Chapter 3](#) features an asymptotic analysis of the number of moves required to solve the different puzzle variations in the worst case. Following this, multiple algorithms for each of the variations are designed by using two algorithmic building blocks. The first approach solves each of the n rows successively from top to bottom. The second approach involves partitioning the tiles into two groups and putting the first group towards the top and the second towards the bottom of the board. We investigate the asymptotic number of moves it takes for the resulting algorithms to solve the puzzle in the worst case. We compare the algorithms by testing them on a large number of randomly generated puzzles in [Chapter 4](#). The testing is performed using a C++ [10] implementation of the proposed algorithms. The C++ project also features an interactive web application where the puzzle can be played and the move sequences calculated by the algorithms can be viewed. This project is included on the accompanying CD ([Appendix A](#)), as well as on GitHub at <https://github.com/Vikplusplus/nx2-puzzle-code>.

2 PRELIMINARIES

We start by introducing a mathematical model of the puzzle and how it is solved (Section 2.1). These concepts are relevant for implementing the algorithms in Chapter 3. Some general features of the puzzle that are shared between all of the three variations dealt with in this work are defined. Where differences occur, we split up the definition for each of the three variations.

2.1 MODELLING THE $n \times 2$ -PUZZLE

The definition of the $n \times 2$ -puzzle is split up into three components. We define the state of the puzzle at any point of solving it, how the puzzle can be manipulated to change its state, as well as the goal of the puzzle.

2.1.1 THE PUZZLE BOARD STATE

The $n \times 2$ -puzzle *board* consists of $2n$ cells. Cells are numbered from 0 to $2n - 1$, as can be seen in Figure 2.1. The puzzle board has n rows and 2 columns, where cell i is located in row $\lfloor i/2 \rfloor$ and in column $i \bmod 2$. Conversely, a cell in row a and column b receives the number $2a + b$.

		Columns	
		0	1
Rows	0	0	1
	1	2	3
	$\vdots$	$\vdots$	
	$n-1$	$2n-2$	$2n-1$

Figure 2.1: The general layout of an $n \times 2$ -puzzle board. Row and column numbers are indicated along the left and top margins of the board respectively. The number of each cell is marked in the top-left corner of the cell.

At any given moment, the puzzle board has a *state* B that defines how each cell is occupied at that moment. The board state is generally denoted using the *cell notation* (except for a special case discussed in [Section 2.2.1](#)). B is denoted in cell notation as an array $B = [c_0, \dots, c_{2n-1}]$, where c_i is a value assigned to cell i . There is always exactly one cell with $c_i = 0$; we say that this cell contains the *empty space*. For all other cells i with $c_i \neq 0$, we say that they contain a *tile* with *label* c_i .

We now introduce the different variations of the $n \times 2$ -puzzle. All variations have boards with the discussed layout, one empty space and $2n - 1$ tiles. The main difference lies in the number of possible label values that the tiles can have. The variations dealt with in this work are:

- The *numbered* $n \times 2$ -puzzle. For each cell i , the tile occupying cell i is labeled with a value $c_i \in \{1, \dots, 2n - 1\}$. Similar to the 15-puzzle, each tile has a unique label, so for any two cells i, j containing tiles, it holds that $c_i \neq c_j$.
- The *2-color* $n \times 2$ -puzzle. Each tile can be thought of as being colored either black or white. The black tiles are assigned the label 1 and the white tiles are assigned the label 2. The board must contain at least one tile labeled 1 and at least one tile labeled 2.
- The *k-color* $n \times 2$ -puzzle, where $k \in \{3, \dots, 2n - 2\}$ is a fixed parameter for the puzzle instance. Again, colors are modeled as numbers. For each cell i , the tile occupying that cell is labeled with a value $c_i \in \{1, \dots, k\}$. For each $c_i \in \{1, \dots, k\}$, there needs to be at least one cell i that contains a tile labeled c_i .

Refer to [Figure 2.2](#) for example board states in each variation.

Numbered	2-color	k -color																														
<table><tr><td>6</td><td>5</td></tr><tr><td>4</td><td>2</td></tr><tr><td></td><td>8</td></tr><tr><td>7</td><td>9</td></tr><tr><td>1</td><td>3</td></tr></table>	6	5	4	2		8	7	9	1	3	<table><tr><td></td><td>1</td></tr><tr><td>2</td><td>1</td></tr><tr><td>1</td><td>2</td></tr><tr><td>2</td><td>2</td></tr><tr><td>1</td><td>2</td></tr></table>		1	2	1	1	2	2	2	1	2	<table><tr><td>1</td><td>1</td></tr><tr><td>4</td><td>3</td></tr><tr><td>3</td><td>1</td></tr><tr><td>4</td><td></td></tr><tr><td>2</td><td>4</td></tr></table>	1	1	4	3	3	1	4		2	4
6	5																															
4	2																															
	8																															
7	9																															
1	3																															
	1																															
2	1																															
1	2																															
2	2																															
1	2																															
1	1																															
4	3																															
3	1																															
4																																
2	4																															

Figure 2.2: Example $n \times 2$ -puzzle board states with $n = 5$ for each variation. From left to right, their cell notations are $[6, 5, 4, 2, 0, 8, 7, 9, 1, 3]$, $[0, 1, 2, 1, 1, 2, 2, 2, 1, 2]$ and $[1, 1, 4, 3, 3, 1, 4, 0, 2, 4]$. Note that $k = 4$ for the k -color puzzle shown here.

2.1.2 MANIPULATING THE PUZZLE

We manipulate a board state B by applying a *move* $\delta \in \{U, D, L, R\}$ to it. Applying a move means sliding a tile into the empty space. Depending on whether a tile is moved up, down, left or right into the empty space, δ has the value U, D, L or R , respectively. A move δ is called *valid for* B if it can be applied to a given board state B . For example, move $\delta = D$ is not valid for a board state that has the empty space in row 0. Tiles cannot be moved diagonally into the empty space. If δ is a valid move for B , $B\delta$ denotes the resulting board state after δ was applied. For a move δ that moved the tile in cell i into the empty space occupying cell j , the values c_i and c_j are swapped in the cell notation of $B\delta$. Refer to [Figure 2.3](#) for an example of a move.

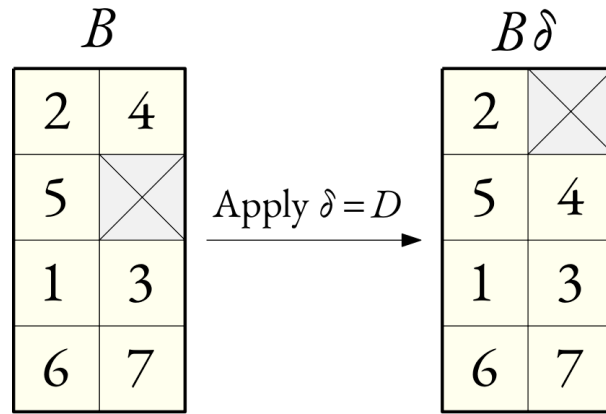


Figure 2.3: On the left, a numbered board state $B = [2, 4, 5, 0, 1, 3, 6, 7]$ is displayed. Applying the move $\delta = D$ makes the tile with number 4 slide downwards into the empty space. This leads to the board state $B\delta = [2, 0, 5, 4, 1, 3, 6, 7]$ depicted on the right.

We can also apply a *move sequence* σ to a given board state B . Move sequences are of the form $\sigma = (\delta_1 \delta_2 \dots \delta_l)$, where each δ_i is a move. Applying σ to B results in the board state $B\sigma$. σ is applied to B by applying $\delta_1, \delta_2, \dots, \delta_l$ in that order. Formally, $B\sigma = B(\delta_1 \delta_2 \dots \delta_l) = (((B\delta_1)\delta_2) \dots)\delta_l$. Similar to simple moves, σ can only be applied to B if it is valid for B ; for $\sigma = (\delta_1 \delta_2 \dots \delta_l)$, this is the case if every δ_i is a valid move for $B(\delta_1 \delta_2 \dots \delta_{i-1})$. Thus, on the puzzle board with 2 columns, a valid move sequence can never contain two ‘ R ’-moves with no ‘ L ’-move inbetween and vice versa. $|\sigma|$ denotes the number of moves in σ , so for $\sigma = (\delta_1 \delta_2 \dots \delta_l)$, it is $|\sigma| = l$. An example of a move sequence is illustrated in [Figure 2.4](#).

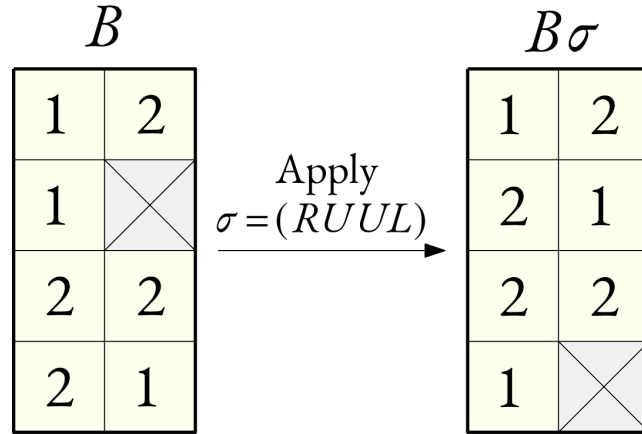


Figure 2.4: On the left, we see the the 2-color variation board state $B = [1, 2, 1, 0, 2, 2, 2, 1]$. Applying the move sequence $\sigma = (RUUL)$ to it results in the board state $B\sigma = [1, 2, 2, 1, 2, 2, 1, 0]$ seen on the right.

2.1.3 THE GOAL OF THE PUZZLE

The goal of the $n \times 2$ -puzzle can now be defined as follows. For a board state B , B^T refers to the *target board state* of B . We want to find a valid move sequence σ , such that $B^T = B\sigma$. Preferably, the size of $|\sigma|$ should be kept as small as possible.

How does B^T result from B ? Naturally, the definition needs to ensure that each tile that exists in B also exists in B^T . More formally, for every tile label value c_i in B , if B has y tiles with the label c_i , then B^T must also have exactly y tiles labeled c_i . Of course, B^T also has exactly one empty space, n rows and 2 columns. In the numbered variation, B^T is fixed for a given n , as each tile label occurs exactly once by definition.

In the following, we define how the tiles of B^T should be arranged. The cell notation of B^T is referred to as $B^T = [c_0^T, \dots, c_{2n-1}^T]$ having the following properties:

- In every variation, the empty space is required to be in the bottom right corner, so $c_{2n-1}^T = 0$.
- All cells $i \in \{0, 1, \dots, 2n - 2\}$ must contain tiles. These tiles should be sorted according to their label in ascending order for all variations. This means that $c_i^T \leq c_j^T$ for all cells i, j containing tiles.

Refer to [Figure 2.5](#) for examples of target boards.

Numbered	2-color	k -color
1	1	1
2	1	1
3	1	2
4	2	3
5	2	3
6	2	4
7	2	4
8	2	4
9	2	4

Figure 2.5: The board states depicted here are the target board states of the board states seen in Figure 2.2. The tiles are ordered left to right, top to bottom. The empty space occupies the cell in the bottom right corner.

Now, with the $n \times 2$ -puzzle defined, the question arises whether the puzzle is always solvable in the different variations or not.

2.2 SOLVABILITY

In the introduction, we saw an example of a 15-puzzle that is not solvable. Can such examples be found for $n \times 2$ -puzzles? We will see that the results differ depending on the variation.

In the following analysis, we say that a board state B' can be *obtained* from B if there is a move sequence σ such that $B' = B\sigma$. Clearly, if B' can be obtained from B , then B can also be obtained from B' . For board states B and B' that can be obtained from one another, we also say that B and B' are *connected*.

We examine the board states B that are connected to the target state B^T . These states are called *solvable*. We first discuss the numbered variation.

2.2.1 NUMBERED VARIATION

Johnson and Story proved that *exactly half* of all board states are solvable for numbered puzzles with n rows and m columns [1]. Later, Archer provided a simpler proof showing that exactly half of the states are solvable in the (numbered) 15-puzzle [11]. Building on Archer's approach, we present a clear and elementary proof for the numbered $n \times 2$ case.

Theorem 2.1. In the numbered $n \times 2$ -puzzle, exactly half of all board states are solvable.

Proof. For ease of presentation, we are going to renumber the cells of the board in this proof. Now, the cells are numbered $1, \dots, 2n$ such that row i contains numbers $2i + 1$ and $2i + 2$. If i is even,

the left cell has the number $2i + 1$; otherwise the right cell does. Visually, the cell numbers increase along a ‘snake-like’ path on the board as seen in [Figure 2.6](#). We refer to this new cell numbering as the *snake-like cell numbering*.

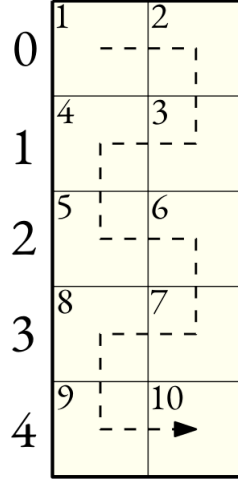


Figure 2.6: The new cell numbers according to the snake-like cell numbering can be seen at the top left of each cell. The cell numbers increase along the dashed arrow with the snake-like shape.

In this proof, board states are described in terms of *slots*. There are only $2n - 1$ slots compared to $2n$ cells; no slot is counted for the cell containing the empty space. The *slot notation* of a board state B is denoted by B_s and is given by an array $B_s = [s_1, s_2, \dots, s_{2n-1}]$. The value s_i describes the position of the tile with label i on the board. For the remaining section, we abbreviate ‘tile with label i ’ and say ‘tile i ’ instead. The value of s_i is defined as follows: Let a cell j be occupied by tile i . If cell j has a lower number than the cell with the empty space, then the slot of tile i is j and $s_i = j$. Otherwise, tile i is in slot $j - 1$ and $s_i = j - 1$ as we skipped counting a slot for the cell containing the empty space.

We use slot notation to facilitate the presentation of the proof; this will become apparent later in the proof. An example for a board state in slot notation can be seen in [Figure 2.7](#).

1 4	2 2
4 6	3 9
5 	6 8
8 1	7 7
9 5	10 3

Figure 2.7: For the slot notation of the depicted board state, we have $s_1 = 7$: The tile 1 is in cell 8, but since the empty space is in cell $5 < 8$, we decrease 8 by 1. The tile 2 is in cell 2 which comes before the empty space, so $s_2 = 2$. In total, the board state has slot notation $[7, 2, 9, 1, 8, 4, 6, 5, 3]$.

As can be seen in [Figure 2.8](#), there are multiple board states sharing the same array in slot notation. This is due to the fact that moving the empty space along the snake-like path leaves the order of the tiles along the path unchanged. For each slot notation array B_s , there are exactly $2n$ board states B that have B_s as their slot notation; that is one for each cell the empty space can occupy. Two board states B, B' with the same slot notation are called *equivalent*. Keeping in mind that moving the empty space along the snake-like path does not affect the slot notation, we can make the following observation:

Observation 2.1. Let B be a board state with slot notation B_s . Let B be connected to a board state B' and let σ be a move sequence with $B\sigma = B'$. In that case, B is also connected to all board states equivalent to B' .

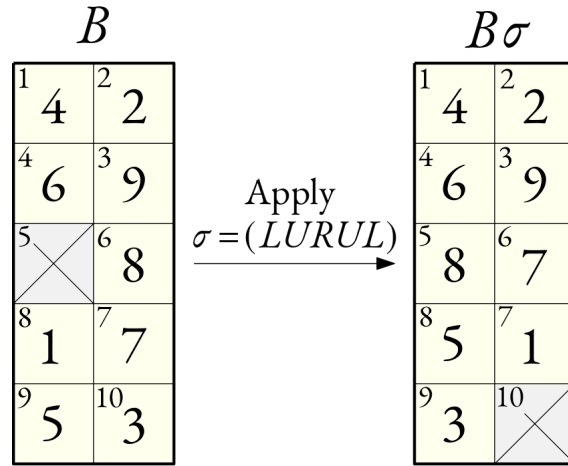


Figure 2.8: The board state on the left is the same as the one from Figure 2.7. It has the slot notation $[7, 2, 9, 1, 8, 4, 6, 5, 3]$ as discussed. Moving the empty space along the snake-like path by applying the move sequence $\sigma = (LURUL)$ to it does not affect the slot notation. The board state to the right also has the slot notation $[7, 2, 9, 1, 8, 4, 6, 5, 3]$ meaning that the depicted board states are equivalent.

In this proof and in Section 2.2.1.1, we use *permutations*. The permutations we work with map a board state to another board state. In this proof, the permutations are permutations of the set of slots, i.e. bijective functions $\pi : \{1, \dots, 2n - 1\} \rightarrow \{1, \dots, 2n - 1\}$. As all permutations in this proof are defined this way, we will omit the explicit definition from now on.

We first recall some basic definitions related to permutations. In the following list, π and τ are permutations and a, b are numbers with $a, b \in \{1, \dots, 2n - 1\}$.

1. Permutations can be denoted using the *cycle notation*. We denote by $a\pi$ the number that a is mapped to under π . An example for a permutation π in cycle notation is $\pi = (1, 3, 2)(4)$: we have $1\pi = 3, 3\pi = 2, 2\pi = 1$ and $4\pi = 4$. Each pair of parentheses denotes a cycle in the permutation. Any fixed numbers can be omitted from the cycle notation, so $(1, 3, 2)(4) = (1, 3, 2)$. The length of a cycle is obtained by counting the numbers in it. A cycle of length ℓ is also called an ℓ -cycle.
2. The *identity permutation*, denoted $()$, is the permutation that fixes every number.
3. Permutations can be multiplied. Multiplication is done left to right. If π, τ are denoted in cycle notation, their product is written as $\pi\tau$. Applying the permutation $\pi\tau$ has the same effect as applying π and τ in that order; formally, it holds that $(a\pi)\tau = a(\pi\tau) = a\pi\tau$. Take $\pi = (2, 4, 1)$ and $\tau = (4, 3, 2)$ as an example. We write $\pi\tau = (2, 4, 1)(4, 3, 2)$. This can be simplified to $\pi\tau = (1, 4)(2, 3)$: for instance, $2(\pi\tau) = 3$ because applying π and τ in

that order means first mapping $2\pi = 4$ and then $4\tau = 3$. We denote by π^k the permutation resulting from multiplying π by itself k times.

4. A *transposition* is a cycle (a, b) of length 2; it swaps two numbers and fixes the rest. Any permutation can be written as a product of transpositions by decomposing its cycles of length greater than 2. For each cycle $(a_1, a_2, a_3, \dots, a_\ell)$ with $a_i \in \{1, \dots, 2n-1\}$, this is done by repeatedly using that $(a_1, a_2, a_3, \dots, a_\ell) = (a_2, a_3, \dots, a_\ell)(a_1, a_2)$.
5. Every permutation π is either *even* or *odd*, depending on whether π can be written as a product of an even or an odd number of transpositions respectively. This means that if π is even, there is no way to write π as a product with an odd number of transpositions (and vice versa). A cycle of length ℓ has the opposite parity to that of ℓ .
6. The inverse π^{-1} has the inverse effect of π : If $a\pi = b$, then $b\pi^{-1} = a$. Therefore, $\pi\pi^{-1} = ()$.
7. The result of *conjugating* π by τ is given by $\tau^{-1}\pi\tau$.

The concept of conjugation becomes relevant later in this proof where we apply the following lemma.

Lemma 2.1. Let π be a permutation in cycle notation so that π consists of only one cycle $(a_1, a_2, \dots, a_\ell)$. Then, conjugating π by another permutation τ results in the permutation $(a_1\tau, a_2\tau, \dots, a_\ell\tau)$.

Proof. We need to prove that $\tau^{-1}\pi\tau = (a_1\tau, a_2\tau, \dots, a_\ell\tau)$. We do this by showing that for every $x \in \{1, 2, \dots, 2n-1\}$, we have

$$x(\tau^{-1}\pi\tau) = x(a_1\tau, a_2\tau, \dots, a_\ell\tau). \quad (1)$$

We consider two cases:

Case 1: $x = a_i\tau$ for some $i \in \{1, \dots, \ell\}$.

Thus, the left side of (1) becomes $(a_i\tau)(\tau^{-1}\pi\tau) = a_i(\tau\tau^{-1})(\pi\tau)$. Since $a_i(\tau\tau^{-1}) = a_i$, we can simplify $a_i(\tau\tau^{-1})(\pi\tau)$ to $a_i(\pi\tau) = (a_i\pi)\tau$. As π maps each a_i to a_{i+1} ($i+1 \bmod \ell$), the left side of (1) is equal to $a_{i+1}\tau$.

For the right side of (1), we have $(a_i\tau)(a_1\tau, a_2\tau, \dots, a_\ell\tau)$. The cycle $(a_1\tau, a_2\tau, \dots, a_\ell\tau)$ maps each $a_i\tau$ to $a_{i+1}\tau$. Thus, the right side of the equation simplifies to $a_{i+1}\tau$ and the equation holds.

Case 2: $x \neq a_i\tau$ for every $i \in \{1, \dots, \ell\}$.

We can first rewrite the left side of (1) with $x(\tau^{-1}\pi\tau) = ((x\tau^{-1})\pi)\tau$. As $(x\tau^{-1}) \neq a_i$, π fixes $(x\tau^{-1})$ and therefore $((x\tau^{-1})\pi)\tau = (x\tau^{-1})\tau = x(\tau\tau^{-1}) = x$.

On the right side of (1), we see that $x(a_1\tau, a_2\tau, \dots, a_\ell\tau) = x$ since the cycle $(a_1\tau, a_2\tau, \dots, a_\ell\tau)$ fixes every $x \neq a_i\tau$.

The equation (1) holds in both cases, thus proving the statement. $\square$

Applying a permutation π to a board state $B_s = [s_1, \dots, s_{2n-1}]$ in slot notation results in the board state $B_s\pi = [s_1, \dots, s_{2n-1}]\pi = [s_1\pi, \dots, s_{2n-1}\pi]$. Note that for any two states $B_s = [s_1, s_2, \dots, s_{2n-1}]$ and $B'_s = [s'_1, s'_2, \dots, s'_{2n-1}]$ in slot notation, there always exists a permutation π with $B_s\pi = B'_s$, namely $s_i\pi = s'_i$ for all slots i .

We can now reduce the problem of finding out whether two board states B, B' are connected to a different problem. With [Observation 2.1](#) in mind, it can be observed that:

Observation 2.2. Let B and B' be board states in their respective slot notations B_s and B'_s . Let π be the permutation with $B'_s = B_s\pi$. B and B' are connected if and only if there is a valid move sequence that causes the same effect as the permutation π .

For the remainder of the proof, we focus on how permutations can be translated to valid move sequences. A problem is that valid moves do not allow swapping arbitrary tiles of the board with each other in the way that transpositions of permutations do. The only transpositions available to us are the ones that correspond to a valid move swapping the empty space with an adjacent tile.

We call a permutation *move permutation* if it corresponds to a valid move on a given board state. Let $\pi_{\{i,j\}}$ with $i, j \in \{1, 2, \dots, 2n\}$ denote the move permutation where i is the cell containing the empty space and j is the adjacent cell from which a tile is moved into cell i . It should be noted that $\pi_{\{i,j\}}$ is still a permutation of *slots*, and that i, j are *cells* that do not denote slots themselves.

We specify the different move permutations. As moving the empty space along the snake-like path does not affect the slot notation, $\pi_{\{i,i+1\}} = ()$. It holds that $\pi_{\{i,j\}} = \pi_{\{j,i\}}^{-1}$, since moving the empty space in a direction and then moving it back does not affect the board state. For odd $i \in \{1, 3, \dots, 2n-3\}$, moving the empty space from cell i downwards causes the permutation $\pi_{\{i,i+3\}} = (i, i+1, i+2)$ as the empty space does not follow the snake-like path in this case. This can be seen in [Figure 2.9](#).

All existing move permutations are summarized in [Table 2.1](#).

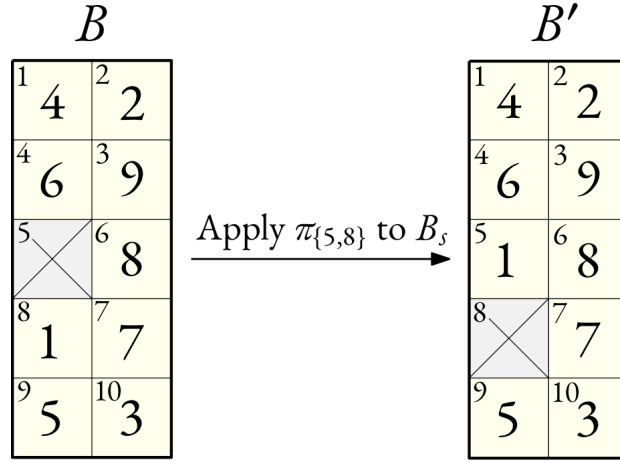


Figure 2.9: The left boards is denoted $B_s = [7, 2, 9, 1, 8, 4, 6, 5, 3]$ in slot notation. Applying $\pi_{\{5,8\}} = (5, 6, 7)$ results in the slot notation $[5, 2, 9, 1, 8, 4, 7, 5, 3]$ of the board on the right.

$\pi_{\{i,i+1\}} = ()$	$\pi_{\{i+1,i\}} = ()$
for $i = 1, 2, \dots, 2n - 2$	
$\pi_{\{i,i+3\}} = (i, i + 1, i + 2)$	$\pi_{\{i+3,i\}} = (i + 2, i + 1, i)$
for $i = 1, 3, \dots, 2n - 3$	

Table 2.1: Every move permutation in cycle notation.

Slot notation is useful in this proof as the move permutations have a very predictable behavior when using it: Every move permutation can always be specified as in [Table 2.1](#), regardless of the tiles that occupy the according cells.

For any move sequence, the permutation it induces can be calculated by multiplying the move permutations corresponding to each move in the order of the moves.

Conversely, any product of move permutations corresponds to a valid move sequence. Although multiplying move permutations $\pi_{\{a,b\}}\pi_{\{c,d\}}$ would imply that the empty space is first moved from cell a to cell b and then from cell c to d , it is not required that $b = c$: Between the two moves, the empty space can be moved to any cell on the board via the snake-like path, without changing the slot notation of the board state.

We determine all permutations that can be obtained by a product of move permutations. These permutations are especially relevant in this proof, since they correspond to a valid move sequence as discussed. Recalling [Observation 2.2](#), we can make the following observation concerning the connectivity of board states:

Observation 2.3. Let π be a permutation resulting from a product of move permutations. Let B be a board state such that the move sequence corresponding to π is valid for B . If B_s denotes the slot notation of B , then B is connected to every board state B' that has the slot notation B'_s with $B'_s = B_s\pi$.

The [Lemmas 2.2](#) to [2.5](#) specify every permutation that can be obtained from multiplying move permutations. We first argue that odd permutations cannot result from such a multiplication.

Lemma 2.2. If τ is an odd permutation, no product of move permutations is equal to τ .

Proof. As τ is odd, it consists of an odd number of transpositions. Meanwhile, all move permutations are even as they are 3-cycles. As any product of even permutations is also even, the product cannot be equal to τ . $\square$

By [Lemma 2.2](#), one half all possible board states cannot be solved in the numbered variation. What about the other half? We prove that every even permutation can indeed be obtained. Taking an arbitrary even permutation τ , we show in 3 steps ([Lemmas 2.3](#) to [2.5](#)) that there is a product of move permutations equal to τ .

Lemma 2.3. If τ is an even permutation, then there exists a product of 3-cycles equal to τ .

Proof. By definition, τ can be written as a product of an even number of transpositions, so we can split the product into pairs of 2 transpositions. Each pair contains either 4, 3 or 2 distinct numbers. We use variables $a, b, c, d \in \{1, 2, \dots, 2n - 1\}$ to refer to these numbers. Each pair of transpositions can be written as a product of 3-cycles. For a pair consisting of

4 distinct numbers a, b, c, d , we have $(a, b)(c, d) = (a, b, c)(a, d, c)$.

3 distinct numbers a, b, c , it holds that $(a, b)(b, c) = (a, c, b)$.

2 distinct numbers a, b , we have $(a, b)(b, a) = ()$. This pair is irrelevant as it can be omitted from τ .

$\square$

Before continuing with [Lemma 2.4](#), *consecutive 3-cycles* are defined as 3-cycles of the form $(i, i + 1, i + 2)$ with $i \in \{1, \dots, 2n - 3\}$.

Lemma 2.4. If τ is an arbitrary 3-cycle, then there exists a product of *consecutive* 3-cycles equal to τ .

Proof. We prove the lemma by induction over k , where τ is defined for numbers $\{1, 2, \dots, k\}$. Similarly to Lemma 2.3, we construct a product of consecutive 3-cycles equal to τ .

Base case ($k = 3$):

The only possible 3-cycles are $(1, 2, 3)$ and $(1, 3, 2)$. Here, $(1, 2, 3)$ is already a consecutive cycle. Further, we have $(1, 3, 2) = (1, 2, 3)^2$, so the statement is true for $k = 3$.

Induction hypothesis:

Any 3-cycle of numbers $\{1, \dots, k\}$ is equal to a product of consecutive 3-cycles for a $k \in \mathbb{N}_{\geq 3}$.

Induction step ($k \rightarrow k + 1$):

Any 3-cycle that does not contain $k + 1$ has only numbers from $\{1, 2, \dots, k\}$, so it can be written as a product of consecutive 3-cycles due to the induction hypothesis.

Any 3-cycle (a, b, c) not containing 1 has only numbers $a, b, c \in \{2, 3, \dots, k+1\}$. By induction hypothesis, there is a product of consecutive 3-cycles that uses only numbers 1 to k and that is equal to $(a - 1, b - 1, c - 1)$. Adding 1 to every number in that product results in a consecutive 3-cycle product for (a, b, c) .

The only remaining 3-cycles are those that contain both 1 and $k + 1$. These are $(1, a, k + 1)$ and $(1, k + 1, a)$ for some $a \in \{1, \dots, k + 1\} \setminus \{1, k + 1\}$. As $(1, k + 1, a) = (1, a, k + 1)^2$, a product of consecutive 3-cycles needs to be constructed only for $(1, a, k + 1)$. For any number $b \in \{1, \dots, k + 1\} \setminus \{1, a, k + 1\}$, it holds that $(1, a, k + 1) = (a, k + 1, b)(1, a, b)$. By applying the induction hypothesis to these permutations, the statement is shown to be true for $k + 1$.

By induction, the statement is obtained for $k = 2n - 1$. This proves the lemma. $\square$

Lemma 2.5. If τ is a consecutive 3-cycle, then τ can be written as a product of move permutations.

Proof. For every consecutive 3-cycle $(1, 2, 3), (2, 3, 4), \dots, (2n - 3, 2n - 2, 2n - 1)$, we need to construct an equal product of move permutations.

If $i \in \{1, 3, \dots, 2n - 3\}$, the move permutation $\pi_{\{i, i+3\}}$ is equal to the consecutive 3-cycle $(i, i + 1, i + 2)$.

If $i \in \{2, 4, \dots, 2n - 4\}$, move permutations can be multiplied to get $(i, i + 1, i + 2)$ as follows.

Consider $\pi_{\{i+1,i+4\}} = (i+1, i+2, i+3)$ (this is an existing move permutation as $i+1$ is odd). Conjugating it by $\pi_{\{i+2,i-1\}} = (i+1, i, i-1)$, [Lemma 2.1](#) can be applied to see that

$$\begin{aligned}\pi_{\{i+2,i-1\}}^{-1} \pi_{\{i+1,i+4\}} \pi_{\{i+2,i-1\}} &= ((i+1)\pi_{\{i+2,i-1\}}, (i+2)\pi_{\{i+2,i-1\}}, (i+3)\pi_{\{i+2,i-1\}}) \\ &= (i, i+2, i+3).\end{aligned}$$

Taking $(i, i+2, i+3)$ and conjugating it by $\pi_{\{i+4,i+1\}} = (i+3, i+2, i+1)$ results in

$$\begin{aligned}\pi_{\{i+4,i+1\}}^{-1} (i, i+2, i+3) \pi_{\{i+4,i+1\}} &= ((i)\pi_{\{i+4,i+1\}}, (i+2)\pi_{\{i+4,i+1\}}, (i+3)\pi_{\{i+4,i+1\}}) \\ &= (i, i+1, i+2).\end{aligned}$$

In total, for even i , $(i, i+1, i+2)$ can be rewritten as a product of move permutations as follows:

$$(i, i+1, i+2) = \pi_{\{i+4,i+1\}}^{-1} \left(\pi_{\{i+2,i-1\}}^{-1} \pi_{\{i+1,i+4\}} \pi_{\{i+2,i-1\}} \right) \pi_{\{i+4,i+1\}}.$$

□

Combining [Lemmas 2.2 to 2.5](#), we can conclude that every even permutation has an equal product of move permutations while every odd permutation has none. As mentioned earlier, every product of move permutations corresponds to a valid move sequence. Keeping [Observation 2.2](#) in mind, we can state:

Theorem 2.2. Let B_s and B'_s be the slot notations of board states B, B' . Let π be the permutation with $B'_s = B_s \pi$. Then, B and B' are connected if and only if π is even.

Since exactly half of all permutations are even, this concludes the proof of [Theorem 2.1](#). □

2.2.1.1 ALGORITHMICALLY DECIDING SOLVABILITY IN THE NUMBERED VARIATION

We aim to design an algorithm that finds out whether a given board state in the numbered $n \times 2$ variation is solvable. Input board states are always encoded in cell notation. Note that the cell notation uses the original cell numbering from [Section 2.1](#).

A simple algorithm based on [Theorem 2.2](#) would calculate the slot notation B_s and B_s^T of the given board state and the target state respectively. It returns true if and only if B_s^T can be obtained from B_s by an even number of transpositions.

Calculating the slot notation of a board states B is a bit inefficient given that B is initially encoded in cell notation. The following argumentation will lead to a more direct approach. First, we examine the simplified case where the empty space is fixed.

Lemma 2.6. Let B and B' be two board states (in cell notation) with the empty space in the same cell x . Let $\pi : \{0, 1, \dots, 2n-1\} \rightarrow \{0, 1, \dots, 2n-1\}$ be the permutation of the cell notation such that $B' = B\pi$. Then, B and B' are connected if and only if π is even.

Proof. Let B_s and B'_s be the slot notations of B and B' respectively. It needs to be shown that the permutation of slots π_s with $B'_s = B_s\pi_s$ is even if and only if π is even.

Both B and B' have the empty space in the same cell x . Thus, π does not permute 0, i.e. $\pi(0) = 0$. In other words, π is a permutation of only the tile labels. We can therefore directly translate π into a corresponding permutation π_s achieving $B'_s = B_s\pi_s$ as follows.

π is first rewritten as a product of transpositions. A transposition (c_i, c_j) of tiles in cell notation with cells $i, j \in \{0, 1, \dots, 2n-1\} \setminus \{x\}$ means swapping the tile occupying cell i with the tile occupying cell j . In slot notation, the same tiles can be swapped by applying the transposition (s_{c_i}, s_{c_j}) as by definition, s_{c_i} and s_{c_j} are the slots containing tiles c_i, c_j respectively. This is illustrated in Figure 2.10.

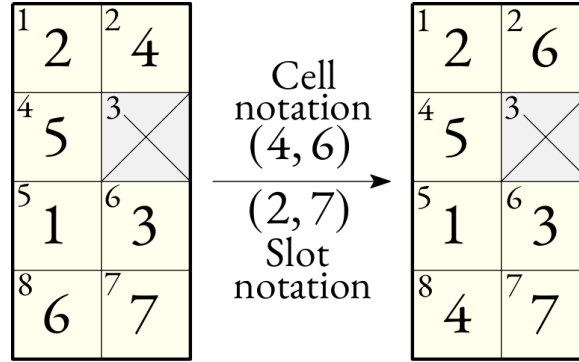


Figure 2.10: From the left board state to right, the tiles with labels 4 and 6 are swapped. In cell notation, we have $[2, 4, 5, 0, 1, 3, 6, 7](4, 6) = [2, 6, 5, 0, 1, 3, 4, 7]$. Meanwhile, tile 4 is in slot 2 and tile 6 is in slot 7, so we use the transposition $(2, 7)$ here. The slot notation from the left board state is $[4, 1, 5, 2, 3, 7, 6]$. The right board state has slot notation $[4, 1, 5, 7, 3, 2, 6]$ which is equal to $[4, 1, 5, 2, 3, 7, 6](2, 7)$.

Every transposition in π therefore corresponds to a transposition in π_s and vice versa. The number of transpositions and especially the parity of π is therefore equal to that of π_s . This proves the lemma. $\square$

Before the final result can be discussed, the following statement first needs to be shown:

Lemma 2.7. Let B, B_α and B_β be board states. Let $\pi_\alpha, \pi_\beta : \{0, 1, \dots, 2n-1\} \rightarrow \{0, 1, \dots, 2n-1\}$ be the permutations of the cell notation arrays such that $B_\alpha = B\pi_\alpha$ and $B_\beta = B\pi_\beta$. Then, the permutation π^* with $B_\beta = B_\alpha\pi^*$ is even if π_α and π_β have the same parity, and odd otherwise.

Proof. We have that $B_\beta = B_\alpha \pi^* = (B\pi_\alpha)\pi^* = B(\pi_\alpha \pi^*)$ and also $B_\beta = B\pi_\beta$. So $\pi_\beta = \pi_\alpha \pi^*$ and π_β must have the same parity as $\pi_\alpha \pi^*$. The parity of $\pi_\alpha \pi^*$ is even if both π_α and π^* have the same parity and odd otherwise. This proves the lemma. $\square$

This leads to the following theorem about the connectivity of arbitrary pairs of board states:

Theorem 2.3. Let B, B_β be two board states and let i, j be the cells containing the empty space in B and B_β respectively. Define $d = \lfloor i/2 \rfloor - \lfloor j/2 \rfloor + |(i \bmod 2) - (j \bmod 2)|$ as the distance between the cells i and j . Let π_β be the permutation that has $B_\beta = B\pi_\beta$. Then, B_β and B are connected if and only if d has the same parity as π_β .

Proof. Take any move sequence σ valid for B such that it moves the empty space from cell i to cell j . For the empty space to be moved into cell j , the number of moves in σ must have the same parity as d , as illustrated by the fact that the empty space can return to a previously occupied cell only after an even number of moves.

Let B_α be the board state that results from applying σ to B , so $B_\alpha = B\sigma$. B_α and B are connected by definition. It remains to show that B_α and B_β are connected.

Let π_α be the permutation with $B_\alpha = B\pi_\alpha$. Every move in σ causes a transposition of two cells, so π_α must have the same parity as σ and d . As both B_α and B_β have the empty space in cell j , it follows from [Lemma 2.6](#) that the two are connected if and only if the permutation π^* with $B_\beta = B_\alpha \pi^*$ is even. As seen in [Lemma 2.7](#), this is true if π_α and π_β have the same parity. This is the case if π_β has the same parity as d as that is also the parity of π_α . This proves the theorem. $\square$

Using [Theorem 2.3](#), the following algorithm checks whether a given board state is solvable:

Algorithm 1: `BOARDSOLVABLE` (B)

Input: A board state $B = [c_0, c_1, \dots, c_{2n-1}]$ in the numbered variation.

Output: A boolean indicating whether B is solvable.

- 1 Find $i \in \{0, 1, \dots, 2n-1\}$ such that $c_i = 0$.
 - 2 $d \leftarrow ((n-1) - \lfloor i/2 \rfloor) + (1 - (i \bmod 2))$.
 - 3 Set $c_i \leftarrow 2n$ in B .
 - 4 Use `QuickSort` on B . While sorting, count the number of swaps y that `QuickSort` does in its calculation.
 - 5 **return** `TRUE` if $(y \bmod 2) = (d \bmod 2)$; else **return** `FALSE`.
-

In line 2, the cell i that contains the empty space is determined. The distance from i to the cell in row $n-1$ and column 1 is calculated in line 3; that is the cell the empty space occupies in B^T . Then, the number $2n$ is assigned to the empty space, so that it is placed at the end of the array

after sorting B with `QuickSort`. The sorted array is equal to B^T . Thus, applying `QuickSort` causes the permutation π with $B\pi = B^T$. Since every swap that `QuickSort` does is equivalent to a transposition, the number of swaps y must have the same parity as π . By [Theorem 2.3](#), `TRUE` is returned if y and d have the same parity and `FALSE` otherwise.

The time complexity of the algorithm is dominated by `QuickSort`. This leads to a worst case running time of $\Theta(n^2)$. Letting `QuickSort` choose the pivot randomly, we achieve an average running time of $\Theta(n \log n)$.

2.2.2 COLORED VARIATIONS

For the 2-color and k -color variations (see [Section 2.1.1](#)), a different result can be observed concerning solvability. With the knowledge from [Section 2.2.1](#), we can show:

Theorem 2.4. For every state B , it holds that B is connected to B^T in the 2- and k -color variations.

We prove a more general statement:

Theorem 2.5. In a 2- or k -colored $n \times 2$ -puzzle, let B and B' be two board states. If it holds that for every label value c_i in B , B and B' have the same number of tiles with that label, then B and B' are connected.

Proof. As B is a board state of a colored puzzle variation, there are at least two cells i, j such that i and j are occupied by tiles with the same label i.e. $c_i = c_j$. Clearly, the same is true for B' .

Similar to the proof of [Theorem 2.3](#), we would like to argue using a permutation π with $B\pi = B'$. Unfortunately, permutations cannot be used on B and B' as the tile labels are not unique.

We transform B into an array $\hat{B}$ where every tile has a unique label: For each cell i containing a tile, we denote the label of that cell as $(c_i)_i$ in $\hat{B}$. $\hat{B}$ is called the *marked board state* of B ; we can think of $\hat{B}$ as being the same as B , only that every tile is marked with the number of the cell it occupied in B . An example of this can be seen in [Figure 2.11](#).

$$\hat{B}$$

⁰ 2	¹ 1
² 3	³ 4
⁴ 4	⁵ 3
	⁷ 1
⁸ 1	⁹ 4

Figure 2.11: On the depicted marked board state $\hat{B}$, every tile is marked in the top left with the number of the cell it occupies. In cell notation, we denote $\hat{B} = [2_0, 1_1, 3_2, 4_3, 4_4, 3_5, 0, 1_7, 1_8, 4_9]$.

Permutations can now be applied to $\hat{B}$. For any permutation π of the tile labels in $\hat{B}$, $\hat{B}\pi$ is a board state that has the same tile labels as $\hat{B}$.

In the board state B' that is to be achieved, tile markings are irrelevant; we only care about whether the original labels match. Therefore, the board states $\hat{B}\pi$ having the marked tile labels can be related to B' as follows: For every cell x occupied by a tile, let y_x be the (marked) tile label of cell x in $\hat{B}\pi$ for any $z \in \{0, 1, \dots, 2n-1\}$. We say that $\hat{B}\pi$ *represents* B' if cell x is occupied by a tile with label y in B' . Refer to Figure 2.12 for an example.

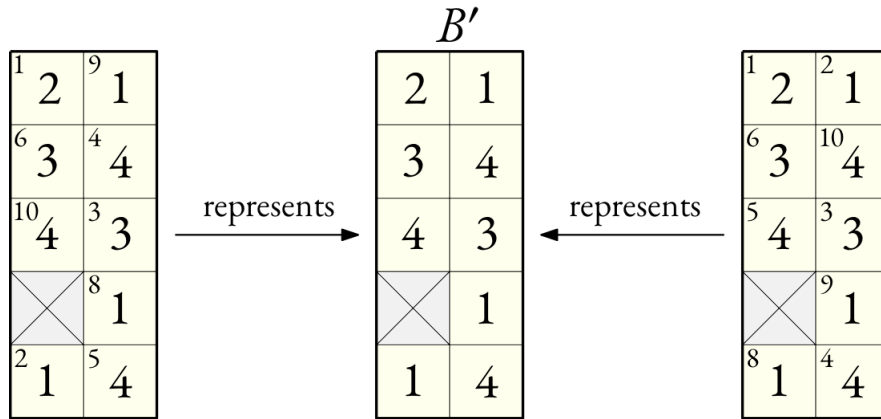


Figure 2.12: The marked board states on the left and on the right represent the board state B' in the center that does not have marked tiles. That is because the original tile labels seen in the center of each tile match.

It can be observed that B and B' are connected if and only if $\hat{B}$ is connected to any board state that represents B' . Clearly, there exists a permutation π such that $\hat{B}\pi$ represents B' .

If the distance d between the empty spaces in B and B' has the same parity as π , [Theorem 2.3](#) can be applied to see that $\hat{B}$ and $\hat{B}\pi$ are connected.

In the case that the parities of d and π are different, let i, j be cells in B such that $c_i = c_j = a$, i.e. the tiles occupying cells i and j have the same label. These tiles have label a_i and a_j in $\hat{B}$. Define $\pi' = \pi(a_i, a_j)$. Then, $\hat{B}\pi' = (\hat{B}\pi)(a_i, a_j)$ represents B' since the only difference between $\hat{B}\pi'$ and $(\hat{B}\pi)$ is that two tiles with the same original label a were swapped. Clearly, π' has the opposite parity of π . Thus, it has the same parity as d , so $\hat{B}$ and $\hat{B}\pi'$ are connected. Thus, $\hat{B}$ is always connected to a board state that represents B' . This proves the theorem. $\square$

Concerning [Theorem 2.4](#), we know that by definition ([Section 2.1.1](#)), B and B^T have the same number of tiles for each label. With [Theorem 2.5](#), it immediately follows that B and B^T are connected.

3 ALGORITHMS

In this chapter, we aim to develop *efficient* algorithms for solving each of the three $n \times 2$ -puzzle variations. What do we mean by efficient? The goal is to minimize the number of moves the proposed algorithms require to solve a puzzle. The input for the algorithms will be a board state denoted in cell notation. In the numbered variation, we assume that the input board state is solvable; any unsolvable board states can be filtered out (using [Algorithm 1](#)). Before starting to design the algorithms, we first analyze how many moves are required at least to solve the puzzles.

3.1 COMPLEXITY OF $n \times 2$ -PUZZLES

For any given board state B , an *optimal move sequence* σ^* is defined as a move sequence achieving $B\sigma^* = B^T$ such that $|\sigma^*|$ is minimal. For a given variation and a fixed number of rows n , the *complexity* is defined as the maximum length of σ^* over all possible puzzles with n rows in that variation. A board state for which σ^* is maximal is called a *worst case board state*.

We saw a complexity result for 15-puzzles in [Section 1.1](#): Brünnger et al. found out that in the 15-puzzle case, there are board states where at least 80 moves are required to solve the puzzle [3]. Exact complexity results are generally difficult to achieve and it usually involves a large amount of processing power; the result by Brünnger et al. was obtained through computations that took “the equivalent of 230 days of (sequential) CPU time” [3] on the NEC Cenju-3 CPU.

In this work, we show an *asymptotic* complexity result. This is done analytically. First, an asymptotic lower bound on the complexity is established for each variation ([Lemma 3.1](#)). This is done by defining a ‘difficult’ board state B^* where any move sequence solving B^* must exceed a certain length. We can define such a board state B^* for every puzzle variation. However, in the 2- and k -color variations, one assumption needs to be made: the number of tiles with each label in B^* is not fixed in advance. For example, the lower bound shown does not apply to a set of puzzles where only one tile is labeled 2 in the 2-color variation. In the k -color variation, there exists a difficult B^* for every $k \in \{3, \dots, 2n - 2\}$. Formally, we show the following statement:

Lemma 3.1. In the numbered, 2-color and k -color variations, let B be a worst case board state. Then, an optimal move sequence σ^* for B has length $|\sigma^*| = \Omega(n^2)$.

Proof. To prove the statement, a difficult board state B^* is defined as follows for each of the variations. For ease of presentation, and without loss of generality, we assume that the number of rows n is even.

1. In the numbered variation, $B^* = [0, 2n - 1, 2n - 2, \dots, 2, 1]$.
2. In the 2-color variation, $B^* = [0, 2, 2, \dots, 2, 1, 1, \dots, 1]$, where exactly $n/2$ of the $2n - 1$ array entries have the value 1.
3. In the k -color variation, B^* is defined as follows: If $k > (3/2)n - 1$, then $B^* = [0, k, k - 1, \dots, 2, 1, 1, \dots, 1]$ so that every value except 1 occurs exactly once in the array. If $k \leq (3/2)n - 1$, B^* is defined such that the last $n/2$ array entries of B^* have the value 1. All previous entries have arbitrary values $a \in \{0, 1, 2, 3, \dots, k\} \setminus \{1\}$ such that every value occurs at least once in B^* .

With the board state B^* defined for each variation, we can make the following observation: In B^* , there are at least $n/2$ tiles which are all at least $n/4$ cells away from their target cell in $(B^*)^T$. These are all the tiles occupying cells i with $i \in I = \{(3/2)n - 1, (3/2)n, \dots, 2n - 1\}$, so the tiles at the bottom of the board. In $(B^*)^T$, each one of these tiles occupies a cell $j \in J = \{1, 2, \dots, n/2\}$ at the top of the board. We examine the minimal distance that the tiles occupying cells $i \in I$ in B have to the cells $j \in J$. By definition, a cell with number a is in row $\lfloor a/2 \rfloor$ (Section 2.1.1). The minimal distance is bound from below by considering only the distance of rows; the tile in cell $(3/2)n - 1$ has the lowest such distance, it is given by:

$$\left\lfloor \frac{\frac{3}{2}n - 1}{2} \right\rfloor - \left\lfloor \frac{\frac{n}{2}}{2} \right\rfloor = \left\lfloor \frac{3}{4}n - \frac{1}{2} \right\rfloor - \left\lfloor \frac{n}{4} \right\rfloor \geq \frac{n}{2} - \frac{n}{4} = \frac{n}{4}.$$

Meanwhile, each move in σ^* can only move one tile by one cell. Clearly, σ^* has to at least include the moves that put the tiles occupying cells $i \in I$ in B to the their target cells $j \in J$. The number of moves in σ^* can therefore be bounded from below by

$$|\sigma^*| \geq |I| \cdot \frac{n}{4} = \frac{n}{2} \cdot \frac{n}{4} = \frac{n^2}{8} \in \Omega(n^2).$$

These arguments can be applied to the case where n is odd with minor adjustments; specifically, in the cases where B^* is defined to have $n/2$ array entries with the value 1, this should be replaced with $\lceil n/2 \rceil$. The definitions of I and J should be changed accordingly. This does not affect the correctness of the results. Thus, the lemma is proven. $\square$

The lower bound shown here coincides with the asymptotic upper bound that is found later in this chapter: For each variation, [Algorithm 8](#) needs $O(n^2)$ moves to solve any input board state. We can therefore conclude:

Theorem 3.1. The asymptotic complexity of each $n \times 2$ puzzle variation is $\Theta(n^2)$.

For the algorithms proposed in the following chapter, this means that we aim to calculate solution move sequences of length $O(n^2)$.

Before developing new algorithms, we examine an algorithm that calculates the optimal move sequence, albeit in exponential time.

3.2 OPTIMAL ALGORITHM

We introduce an optimal algorithm that follows a simple brute force approach. Brute force algorithms are prone to being quite resource heavy. Although the optimal algorithm calculates a move sequence of minimal length, it is infeasible in most cases; even for small boards in the numbered variation with $n = 6$, the algorithm takes an impractical amount of time and storage. Still, implementing the optimal algorithm is helpful in multiple aspects: for smaller boards, it can serve as a benchmark to which other algorithms can be compared. Additionally, by observing the optimal move sequences calculated by the algorithm, specific move patterns can be found and adopted in other algorithms. The optimal algorithm can also be used for solving small parts of the board in other algorithms.

We first analyze how the optimal solution can be calculated using ‘brute force’. The optimal algorithm is referred to as OPT. Refer to [Algorithm 2](#) for the according pseudocode.

The strategy of OPT is as follows: Given an input board state B , OPT checks, for every move δ that is valid for B , whether $B\delta = B^T$. If B^T was not found, OPT will visit each $B\delta$ and again check whether applying a valid move results in B^T . If OPT reaches a board state that it has visited before, that board state is ignored. This process is repeated until B^T is found. OPT then backtracks to reconstruct the solution move sequence.

OPT is a *search algorithm*. As OPT tries out every possible solution until the target state is found, it conducts an *exhaustive search*. Search algorithms are very common for finding solutions to puzzles and games in Artificial Intelligence. Algorithms similar to OPT can be used to solve Chess puzzles or Connect Four for example. Search algorithms generally traverse a *search tree* using an appropriate search method. The search tree is usually not calculated as a whole before starting the traversal. Instead, it is successively constructed while the search algorithm runs, starting from the

root.

In our case, the search tree is induced by the following rules. Let B be the input board state for OPT.

- For every board state B' that is connected to B , there exists exactly one node $v_{B'}$ in the search tree. Note that this includes the node v_B .
- v_B is the root of the search tree.
- There exists a *directed* edge (v_{B_1}, v_{B_2}) if there is a move δ that is valid for B_1 and has $B_1 \delta = B_2$. Additionally, v_{B_2} needs to have a larger depth than v_{B_1} , as otherwise, v_{B_2} would have been visited before v_{B_1} .

The following Figure 3.1 illustrates the upper part of a search tree.

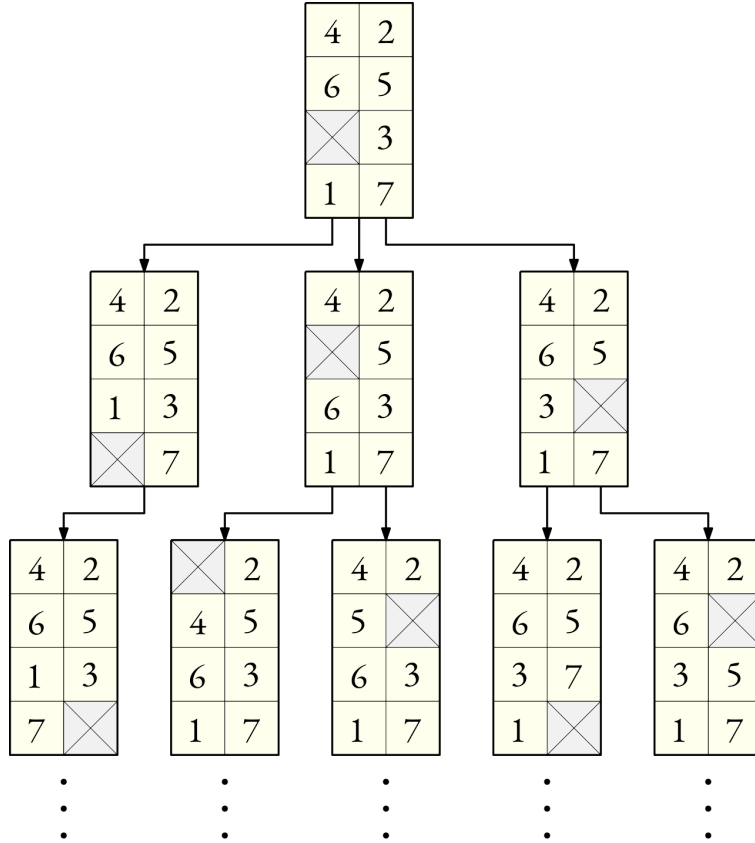


Figure 3.1: The depicted search tree has the input board state as its root. For every valid move, the board states have the resulting board state as a child node. Board states like the one on the left in the middle row have only one outgoing edge as the empty space occupies a corner cell.

The size of the search tree determines how many nodes OPT has to visit before finding the target state in the worst case. Depending on the variation, the search tree has the following size:

- In the numbered variation, there are $(2n)!$ possible board states. As established in [Theorem 2.2](#), any board state is connected to exactly half of all possible board states. As a consequence, the search tree consists of $(2n)!/2$ nodes.
- In the 2-color variation, let $y_1, y_2 \in \{1, 2, \dots, 2n - 2\}$ be the number of tiles with labels 1 and 2 respectively. y_1 and y_2 must fulfill $y_1 + y_2 = 2n - 1$ as one cell needs to be left unoccupied for the empty space. As seen in [Theorem 2.5](#), every board state is reachable, so the multinomial coefficient can be used to get $(2n)!/(y_1! \cdot y_2!)$ as the size of the search tree.
- In the k -color variation, let $y_1, y_2, \dots, y_k \in \{1, 2, \dots, 2n - 2\}$ be the number of tiles with labels $1, 2, \dots, k$ respectively. Similarly to the 2-color variation, $y_1, y_2, \dots, y_k$ need to have $\sum_{i \in \{1, 2, \dots, k\}} y_i = 2n - 1$. Again, every board state is reachable and the number of nodes in the search tree can be expressed as $(2n)!/(\prod_{i \in \{1, 2, \dots, k\}} y_i!)$. This number generally gets *smaller* as k decreases. This occurs because the average size of the y_i is larger for smaller k , which has a significant impact since y_i grows factorially, i.e. as $y_i!$.

This means that the size of the search tree grows exponentially with n in each variation. In the numbered variation for $n = 7$, OPT would have to visit 43, 589, 145, 600 nodes in the worst case; it is clear that OPT quickly becomes infeasible for larger n .

If B and B^T are connected by a move sequence of length d , it can take OPT an exponential number of visited nodes before finding B^T . The number of nodes with depth d grows exponentially in d . For any board state B , there are either 2 or 3 valid moves. This depends on whether the empty space occupies a cell situated in one of the 4 corners or not. As one of the valid moves is always the move leading back to the parent of v_B , v_B has either 1 or 2 child nodes, if it is not a leaf. The majority of cells are not in a corner of the board, so we can expect the number of nodes in depth d to grow exponentially with base 2.

With the search tree described, it remains to choose how OPT should navigate it. As the goal is to find the shortest solution move sequence, Breadth-first search (BFS) is the appropriate choice. Using BFS, we make sure that OPT has visited every node in depth d before continuing with depth $d + 1$. We now describe OPT. OPT is the same for every variation.

As we saw, OPT quickly becomes infeasible for larger n . In the remainder of the chapter, we design algorithms that are capable of handling boards of larger size without difficulty. We start by

Algorithm 2: OPT (B)

Input: A board state $B = [c_0, c_1, \dots, c_{2n-1}]$.

Output: An optimal move sequence σ such that $B\sigma = B^T$.

```
1 Determine the cell notation of  $B^T$ .
2 Initialize a queue  $Q$  and a set  $S$  that both contain only  $B$ .
3 while  $Q$  is not empty do
4   Let  $B^Q$  be the first element in  $Q$ . Remove  $B^Q$  from  $Q$ .
5   if ( $B^Q = B^T$ ) then
6     break.
7   else
8     for  $\delta \in \{U, D, L, R\}$  such that  $\delta$  is valid for  $B^Q$  do
9       Calculate  $B^+ \leftarrow B^Q \delta$ .
10      if  $B^+ \notin S$  then
11        Add a pointer from  $B^+$  pointing to  $B^Q$ .
12        Add  $B^+$  to  $S$  and enqueue  $B^+$  to  $Q$ .
13 Let  $B^*$  be the board state last removed from  $Q$  (so  $B^* = B^T$ ).
14 Initialize an empty move sequence  $\sigma$ .
15 while  $B^* \neq B$  do
16   Let  $\hat{B}$  be the board state that  $B^*$  points to. Let  $\delta$  be the move with  $B^* = \hat{B}\delta$ .
17   Append  $\delta$  to  $\sigma$  such that  $\delta$  is now the first move in  $\sigma$ .
18   Set  $B^* \leftarrow \hat{B}$ .
19 return  $\sigma$ .
```

designing 2 algorithmic building blocks that will be essential to the full algorithms developed in [Section 3.5](#).

In the remaining chapter, when comparing two rows r_1, r_2 , we often say that row r_1 is *higher* or *lower* than row r_2 . This refers to the location of the rows on the board, not their numbering: If row r_1 is higher than row r_2 , this means that $r_1 < r_2$.

3.3 solveRow-PROCEDURE

Given a board state B and a row r , the solveRow-procedure solves row r . A cell i is called *solved* in B if $c_i = c_i^T$. Consequently, a row r is called solved when both its cells $2r$ and $2r + 1$ are solved. When calling solveRow on row r , the rows above r are assumed to be solved already; this will become apparent in [Section 3.5](#), where the full algorithms using solveRow are introduced. solveRow does not involve moves that move the empty space into the rows above r .

The general approach to solving a row is the same across all variations. We are given a row $r \in \{0, 1, \dots, n-3\}$ and a board state B . The rows $0, 1, \dots, r-1$ of B are already solved. The empty space is in row r or in a row below it.

The row r is generally solved as follows. In the algorithms described in the remaining chapter, the input board state B is a dynamic variable that changes during the execution of the algorithm, according to the moves that are applied to it.

Algorithm 3: `solveRow` (B, r)

Input: A board state B and a row r .

- 1 Let $B^T = [c_0^T, c_1^T, \dots, c_{2n-1}^T]$. Select a tile a with label c_{2r}^T in B .
 - 2 Move tile a into cell $2r+1$ using the `liftTile`-procedure.
 - 3 Select a tile b with label c_{2r+1}^T in B .
 - 4 Apply `liftTile` to move tile b into cell $2(r+1)+1$.
 - 5 Move the empty space to cell $2r$.
 - 6 Apply the move sequence (LU) .
-

In the 2-color and k -color variation, when selecting tiles a and b , there can be multiple tiles with the target label. In that case, `solveRow` selects the tile that is in the highest row $r' \geq r$. In line 2, `solveRow` places tile a in row r , but in the opposite column of where it belongs. Then, tile b is placed in the cell directly below tile a , such that moving the empty space to cell $2r$ and then applying (LU) solves row r . An example showcasing this approach for numbered puzzles is displayed in [Figure 3.2](#).

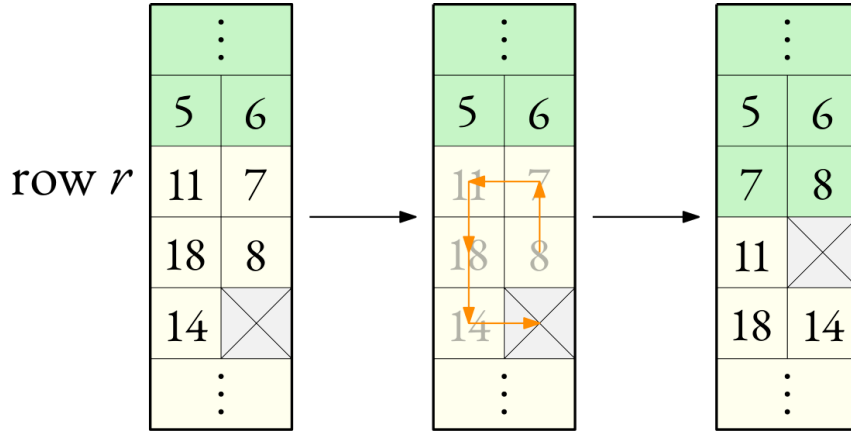


Figure 3.2: In the depicted board states, the rows colored in green have already been solved. The row r that is being solved contains the tiles with labels 7 and 8 in B^T . In the board state on the left, `solveRow` just executed line 4; tiles 7 and 8 now occupy the specified cells. In line 5 and 6, the move sequence highlighted in the central board state is applied. The tiles are moved into the empty space, following the orange arrows. This leads to the board state on the right, where row r is solved.

The `liftTile`-procedure is used on the tiles a and b in lines 2 and 4 of `solveRow` respectively. Generally, for a tile y , `liftTile` works as follows:

Algorithm 4: `liftTile` (B, r, y)

Input: A board state B , a row r and a tile y .

- 1 **if** tile y occupies a cell i in column 0 **then**
 - 2 Move the empty space into cell $i + 1$.
 - 3 Apply the move L .
 - 4 **while** tile y is not in row r **do**
 - 5 Move the empty space into the cell directly above tile y .
 - 6 Apply U to move tile y up one row.
-

One iteration of the while-loop in line 4 of `liftTile` is illustrated in [Figure 3.3](#).

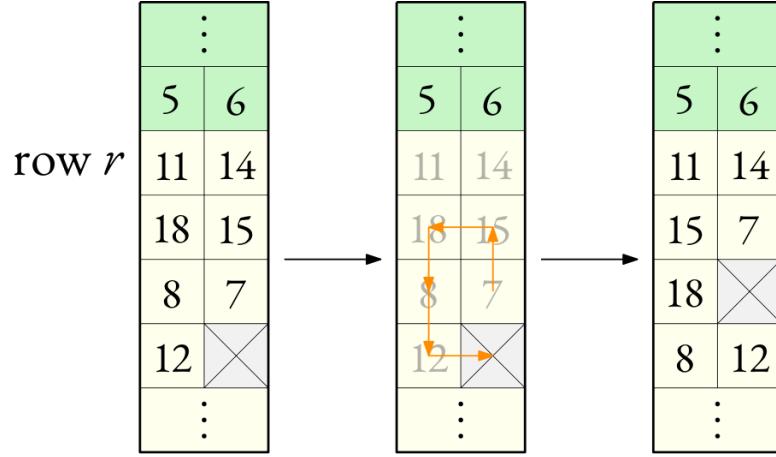


Figure 3.3: In the left board state, tile 7 is the tile that `liftTile` is applied on. It should be moved to the cell in column 1 in row r . The move sequence highlighted in the central board state is executed in lines 5 and 6. This results in the board state seen on the right, where tile 7 is one row further up compared to before.

There are some exceptions where the `solveRow`- and `liftTile`-procedures are not applicable as they are defined now. Naturally, when calling `solveRow` on a row r that is already solved, the procedure should not apply any moves to the board state. If row r already contains the correct tiles but the tiles are in opposite order, row r is solved by first moving the empty space into cell $2(r + 1)$ and then applying the move sequence (*DLUURDLDRUULDRDLU*). `solveRow` cannot be used on rows $n - 2$ and $n - 1$ because of limited board space. These rows can be solved with `OPT` instead.

We briefly discuss how many moves the `solveRow`-procedure requires in the worst case. When calling `solveRow` on row r , the part of the board where the empty space is moved around is $(n - r)$ rows large. Thus, in `liftTile`, moving the empty space next tile y in line 2 takes $O(n - r)$ moves. Lifting tile y up by one row (lines 5 and 6) takes a constant number of moves. Thus, when tile y reaches row r , an additional $O(n - r)$ moves were used. Every exception can also be handled with $O(n - r)$ moves. Thus, `solveRow` solves row r using $O(n - r)$ moves.

3.4 partition-PROCEDURE

The `partition`-procedure presented here is mostly based on a similar algorithm proposed by Zhong in 2023 [9]. In this work, we add small improvements and provide a more detailed explanation of the procedure.

`partition` separates the tiles of the board according to a color that each tile has been assigned before. Before calling `partition`, one group of tiles is assigned the color *black*, all other tiles are assigned the color *white*. `partition` then accomplishes the following:

Let q be the number of black tiles on a given board B . `partition` applies a move sequence σ to B , such that every cell in $\{0, 1, \dots, q - 1\}$ is occupied by a black tile in $B\sigma$. Every cell with number $i > q - 1$ contains either a white tile or the empty space. Refer to the board state on the right in Figure 3.4 for an example.

Given a board state with an assignment of a color to each tile, `partition` is executed in 3 phases ([Algorithms 5 to 7](#)). An illustration of what each phase accomplishes is shown in Figure 3.4.

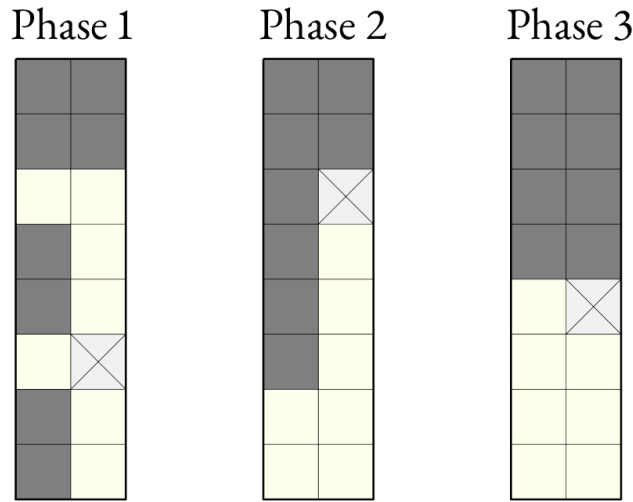


Figure 3.4: An illustration of how the board state has changed after each phase during the execution of `partition`. When Phase 3 is completed, the tiles are completely separated.

In each phase, *cyclic shifts* of tiles involving the tiles between an upper row t and a lower row b are applied repeatedly. The cyclic shifts differ from phase to phase, but the general approach for each phase is to repeatedly determine rows t, b and apply the cyclic shift. This is done until the board state property that the phase aims for is established.

Phase 1 accomplishes that the resulting board state has the following property: Let row u be the lowest row that has a black tile in column 1. Then, every cell $i \in \{0, 1, \dots, 2u + 1\}$ is occupied by a black tile. Refer to Figure 3.4 above for an example.

In phase 1, the `partition`-algorithm moves black tiles from column 1 to column 0. If row r is the highest row containing a white tile (row r can change during the execution of phase 1), then all black tiles that are in row r or below and in column 1 are moved to column 0 in phase 1.

In detail, phase 1 consists of the following steps:

Algorithm 5: `partition-phase1` (B, A)

Input: A board state B and an assignment A assigning a color to each tile.

```

1  if the empty space is in column 0 then
2    └─ Move the empty space to column 1.
3  Let row  $r$  be the highest row containing a white tile.
4  Initialize row  $b \leftarrow r$ .
5  while cell  $2b + 1$  does not contain a black tile and  $b \leq n - 1$  do
6    └─  $b \leftarrow b + 1$ .
7  if the empty space is in a row above row  $r$  then
8    └─ Move the empty space down to row  $r$ .
9    └─ if a white tile was moved to the row above row  $r$  then
10      └─  $r \leftarrow r - 1$ .
11 else
12   └─ if the empty space is in a row below row  $b$  then
13     └─ Move the empty space up to row  $b$ .
14     └─  $b \leftarrow b + 1$ .
15 while  $b \neq n$  do
16   Initialize row  $t$  as the row directly above row  $r$ .
17   while row  $t + 1$  does not contain any black tiles do
18     └─  $t \leftarrow t + 1$ .
19   Move the empty space up to row  $t$ .
20   Switch the column of the empty space.
21   Move the empty space down to row  $b$ .
22   Switch the column of the empty space.
23   while cell  $2b + 1$  does not contain a black tile and  $b \leq n - 1$  do
24     └─  $b \leftarrow b + 1$ .
25   └─ Update row  $r$  if necessary.

```

In phase 1, row b refers to the row of the highest black tile that needs to be moved to column 0. If row r is the highest row containing a white tile, then row $t + 1$ is the highest row, which is on the same level as row r or below, that contains a black tile. Before the while-loop in line 11, the algorithm moves the empty space into the required position for the cyclic shifts applied in the while-loop in lines 15 to 18. When there are no more black tiles that need to be moved to column 0, b is assigned the value n and phase 1 ends. An example cyclic shift used in phase 1 is depicted in [Figure 3.5](#).

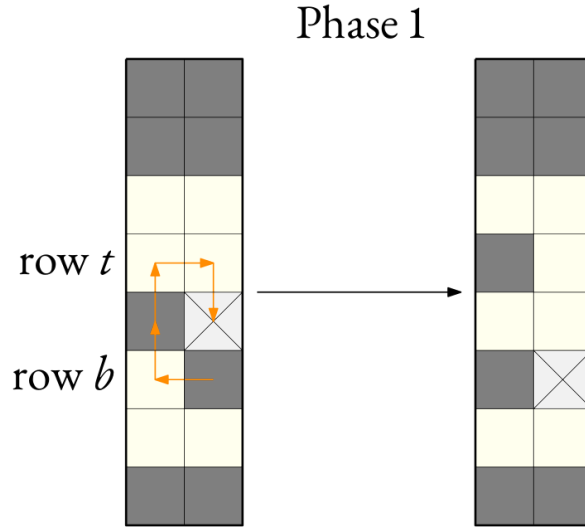


Figure 3.5: An example for the cyclic shift applied in phase 1 of `partition`. The tiles are moved into the empty space in the direction of the orange arrows, resulting in the board state on the right.

There is one exception that can occur in phase 1. If b is the highest row containing a white tile, then $t + 1$ and b are the same number when applying the cyclic shift. In that case, the empty space is simply moved to row b instead of applying the cyclic shift. Afterwards, the algorithm continues normally.

When phase 2 begins, the board state has the property established by phase 1. The empty space is in column 1 below the lowest row that has a black tile in column 1. After phase 2, the property that was established by phase 1 is maintained and a new property is added: The board state has only white tiles in cells $i \in \{2d, 2d+1, 2n-1\}$, where row d denotes the highest row that contains a white tile in column 0. This property is symmetric to the property of phase 1. Refer to [Figure 3.4](#) for an example.

In phase 2, `partition` moves white tiles from column 0 to column 1. If row r is the lowest row containing a black tile (row r can change during phase 2), then all white tiles that are above row r and in column 0 are shifted to column 1.

Phase 2 is executed as follows:

Algorithm 6: partition-phase2 (B, A)

Input: A board state B with the property established by phase 1 and an assignment A assigning a color to each tile.

- 1 Initialize row b as the lowest row containing a black tile.
 - 2 **while** column 0 contains a white tile in a row above b **do**
 - 3 Initialize row $t \leftarrow b - 1$.
 - 4 **while** row t does not contain a white tile in column 0 and $t > 0$ **do**
 - 5 $t \leftarrow t - 1$.
 - 6 Move the empty space up to row t .
 - 7 Move the empty space left to column 0.
 - 8 Move the empty space down to row b .
 - 9 Move the empty space right to column 1.
 - 10 Move the empty space up to row t .
 - 11 $b \leftarrow b - 1$.
-

Row b is defined in line 1. Row t is the first row above row b that contains a white tile in column 0. The cyclic shift is applied in rows 7 to 10. Each cyclic shift moves the lowest black tile one row up, so b is decreased by 1 in row 11. An example for the cyclic shift used in phase 2 is shown in [Figure 3.6](#).

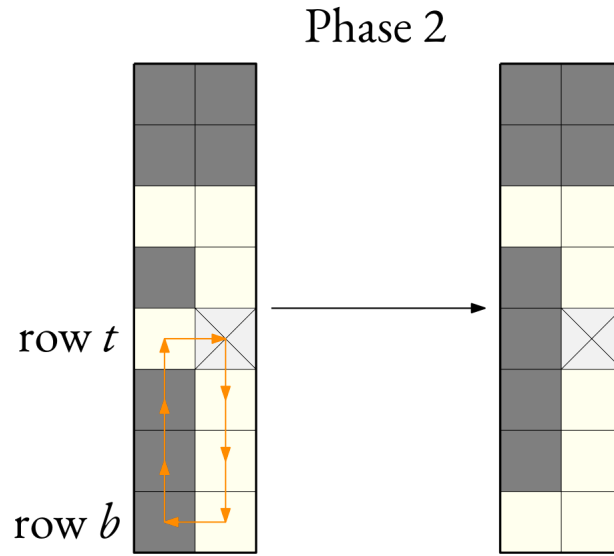


Figure 3.6: An example for the cyclic shift applied in phase 2 of partition. The tiles are moved into the empty space in the direction of the orange arrows, resulting in the board state on the right.

At the beginning of phase 3, the empty space is in column 1 and the board state has the property achieved by phase 1 and phase 2. After phase 3, the black and white tiles are separated as seen in Figure 3.4 and partition is completed.

Phase 3 moves the black tiles closer towards the ‘block’ of black tiles at the top of the board. That is accomplished by the following steps:

Algorithm 7: partition-phase3 (B)

Input: A board state B with the properties established by phase 1 and 2.

- 1 Initialize row b as the lowest row containing a black tile.
 - 2 Initialize row t as the highest row containing a white tile.
 - 3 **while** $t \neq b + 1$ **do**
 - 4 Move the empty space left to column 0.
 - 5 Move the empty space down to row b .
 - 6 Move the empty space right to column 1.
 - 7 Move the empty space up to row $t + 1$.
 - 8 $t \leftarrow t + 1$.
 - 9 $b \leftarrow b - 1$.
-

Rows b and t are defined in lines 1 and 2 respectively. The cyclic shift is applied in rows 4 to 7. This moves the lowest black tile up and the highest white tile down by one row, t and b are adjusted accordingly. An example execution of the cyclic shift of phase 3 is illustrated in Figure 3.7.

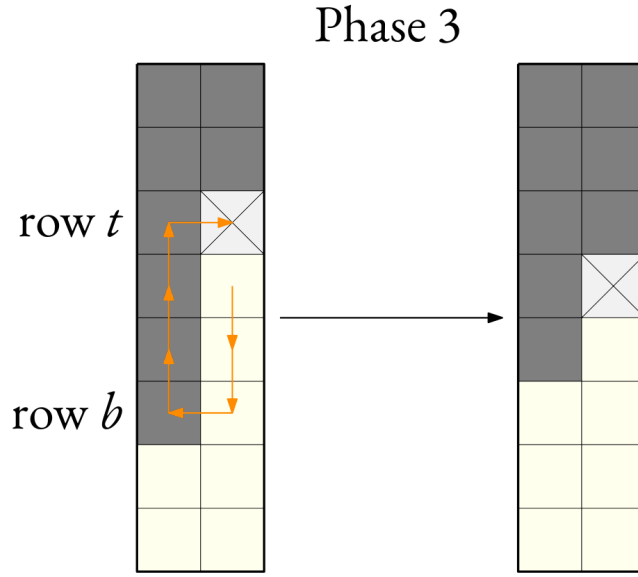


Figure 3.7: An example for the cyclic shift applied in phase 3 of `partition`. The tiles are moved into the empty space in the direction of the orange arrows, resulting in the board state on the right.

We conclude the discussion of `partition` by establishing an asymptotic upper bound on the number of moves the procedure makes in the worst case. In each phase, the cyclic shifts dominate the number of moves required. The number of moves needed in any of the cyclic shifts can be generously bounded by the following argument. Rows b and t are always at most $n - 1$ rows apart from each other, so in each cyclic shift, $O(n)$ moves are made. Afterwards, the values of b and/or t are updated such that the step is repeated at most $n - 1$ times. Therefore, each of the 3 phases requires $O(n^2)$ moves and the same upper bound applies to `partition`.

For the remaining chapter, we use the `solveRow`- and `partition`-procedures as building blocks to design algorithms that efficiently solve $n \times 2$ -puzzles.

3.5 ALGORITHMS

We propose multiple algorithms for each variation of the $n \times 2$ -puzzle; three for the numbered variation and two for the 2-color and k -color variations respectively.

In each case, the first algorithm relies mainly on the `solveRow`-procedure. We describe the algorithm in a unified manner for all variations, as the differences between them are only minor.

3.5.1 RowByRow

The `solveRow`-procedure solves one row of the board. Logically, solving entire puzzles with the procedure can be done by iteratively using `solveRow` on every row of the board; we follow this approach to design the RowByRow-algorithm as follows.

Algorithm 8: RowByRow (B)

Input: A board state B .

- 1 **for** $r \leftarrow 0$ **to** $n - 3$ **do**
 - 2 Use `solveRow` (B, r).
 - 3 Solve rows $n - 2$ and $n - 1$ with OPT.
-

Recall that after a row r is solved by `solveRow`, calling `solveRow` on a row r' below r will not impact the tiles of row r . The tiles in a solved row r will stay in their cells until RowByRow is finished. The bottom rows $n - 2$ and $n - 1$ are solved with OPT. When OPT is completed, the target state is obtained.

The number of moves required in the 2- and k -color variations can be reduced further. For an input board state B , the `solveRow`-procedure is applied only until reaching the highest row r^* such that r^* contains a tile with label 2 (respectively k) in B^T . After solving that row, the target state is obtained by simply moving the empty space to cell $2n - 1$.

In the k -color variation, the RowByRow-algorithm needs to be adjusted slightly as it could fail otherwise. This can be seen in the following example: For $k = n - 2$, let B be a board state that has two tiles with label 1 while for every other label $2, 3, \dots, k$ there is only one tile with that label. Recall the proof of [Theorem 2.5](#); B can be solved, but only if the two tiles with label 1 are placed the right way around. If the two tiles are not placed correctly, the last rows of the board cannot be solved.

How can this issue be dealt with in the k -color variation? In the target state B^T , we determine the highest numbered pair of cells $i, i + 1$, such that cells i and $i + 1$ contain tiles with the same label a . RowByRow is executed as usual, until the point where the algorithm calls `solveRow` on the row with cell i . The algorithm checks whether the board state can be solved (see [Algorithm 1](#)) for the possible orders of the two tiles labeled a . The tiles are put in the correct order such that the board state can be solved. Then, the remaining rows are solved as usual.

Since each call of `solveRow` causes $O(n)$ moves to be made, the `RowByRow`-algorithm makes $O(n^2)$ moves. The special case mentioned for k -color is handled in a constant number of moves.

For the remaining chapter, every algorithm described uses the `partition`-procedure. In the numbered variation, we present two such algorithms.

3.5.2 NUMBERED VARIATION: `DIVIDEONCE`

This algorithm combines `partition` and `RowByRow`. First, `partition` is used to move each tile with a label $a \leq n$ into the upper half of the board and all other tiles into the lower half. Intuitively, each tile is ‘presorted’ into the half of the board that it belongs to. Afterwards, `RowByRow` is used.

Algorithm 9: `DIVIDEONCE` (B)

Input: A board state B in the numbered variation.

- 1 Let A be an assignment of the tiles in B .
 - 2 **foreach** tile in B **do**
 - 3 **if** the tile has a label $a \leq n$ **then**
 - 4 Assign the tile the color black in A .
 - 5 **else**
 - 6 Assign the tile the color white in A .
 - 7 Use `partition` on B according to A .
 - 8 Use `RowByRow` (B).
-

The `partition`-procedure places all tiles with labels $a \leq n$ into cells $i \leq n$. Similarly, tiles with labels $a > n$ are placed in cells $i > n$. The maximal distance that a tile can have from its cell in the target state is approximately halved. Therefore, the maximal number of moves required by `RowByRow` to solve each row is also about half as much as before.

The number of moves used by `DIVIDEONCE` can be slightly decreased by using an *inverted* version of `partition`. This version achieves the same result as the original, except that the empty space is placed among the black tiles. The inverted version can be easily implemented; in phase 1, the inverted version moves white tiles from column 0 to column 1 whereas the original version moves black tiles from column 1 to column 0. Phases 2 and 3 are altered in the same way in the inverted `partition`-procedure. The inverted version of `partition` places the empty space in the upper half of the board, which is beneficial since `RowByRow` solves the rows from top to

bottom. The inverted version of partition is necessary in the following algorithm.

Both partition and RowByRow each make $O(n^2)$ moves. In DIVIDEONCE, they are applied successively. Therefore, DIVIDEONCE requires $O(n^2)$ moves to solve the worst case puzzle.

3.5.3 NUMBERED VARIATION: DIVIDEANDCONQUER

Following the approach of DIVIDEONCE, the DIVIDEANDCONQUER-algorithm divides the board iteratively until each tile only has a constant distance to its target cell. DIVIDEANDCONQUER can be thought of as a *recursive* algorithm; the board is divided into two smaller *subboards* of about equal size. The subboards can be solved *mostly* independent of each other. If a subboard has reached a size of three rows or less, it is solved with the solveRow-procedure. The algorithm always completely solves the upper subboard before continuing with the lower subboard, as can be seen in Figure 3.8.

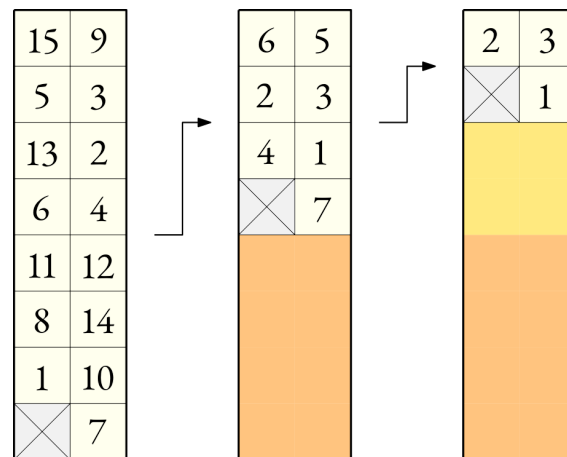


Figure 3.8: Shown here are the first two steps of the DIVIDEANDCONQUER-algorithm on the example board state seen left. First, the board is partitioned two times, splitting it up into three subboards indicated by the different colors. The resulting board state seen on the right has a subboard with two rows at the top. This subboard is solved with solveRow next. DIVIDEANDCONQUER is then applied to the remaining subboards from top to bottom, starting with the yellow subboard.

Each subboard has a top row t and a bottom row b . Every tile in the rows from t to b , except for one, has its target cell in these rows. As the bottom row b of a subboard indicates the top row $t = b + 1$ of the subboard below it, the algorithm uses a linked list L to represent the subboards. Two consecutive numbers l, l' in L indicate the top and the bottom row of one subboard.

We now give a detailed description of the `DIVIDEANDCONQUER` algorithm.

Algorithm 10: `DIVIDEANDCONQUER` (B)

Input: A board state B in the numbered variation.

```

1 Initialize a linked list  $L$  with the numbers 0 and  $n - 1$ . The number 0 is at the front of  $L$ .
2 Let  $l_1$  and  $l_2$  be the first and the second number of  $L$  respectively.
3 while TRUE do
4   if  $(l_2 - l_1 \leq 2)$  then
5     if  $(l_2 = n - 1)$  then
6       break.
7     for row  $r \leftarrow l_1$  to  $l_2$  do
8       Use solveRow on row  $r$ .
9     Remove  $l_1$  from  $L$ .
10    Update  $l_1$  and  $l_2$ .
11  else
12    Let  $B'$  be the subboard of  $B$  bound by  $l_1$  and  $l_2$ .
13    Let  $n' \leftarrow l_2 - l_1 + 1$  be the amount of rows of  $B'$ .
14    Let  $A$  be an assignment of the tiles in  $B'$ .
15    foreach tile in  $B'$  do
16      if the tile has a label  $a \leq 2l_1 + n' - 1$  then
17        Assign the tile the color black in  $A$ .
18      else
19        Assign the tile the color white in  $A$ .
20    if  $n'$  is odd then
21      Switch the assignment of the tile labeled  $2l_1 + n'$  to black in  $A$ .
22    Use the inverted version of partition on  $B'$  according to  $A$ .
23    Let row  $r$  be the lowest row that contains a black tile in  $B'$  after partitioning.
24    Insert  $r$  into  $L$  as the successor of  $l_1$ .
25    Set  $l_2 \leftarrow r$ .
26 Solve the rows  $l_1$  to  $n - 1$  with OPT.

```

The case in line 4 where $l_2 - l_1 \leq 2$ occurs when the subboard bound by l_1 and l_2 has 3 rows or less. This case is equivalent to the ‘base case’ if `DIVIDEANDCONQUER` is thought of as a recursive algorithm. If the bottom row l_2 is equal to $n - 1$, the bottom of the board is reached and the while-loop is broken. If $l_2 \neq n - 1$, the subboard is solved by using `solveRow` on every row.

If the subboard is larger than 3 rows, the else-case in line 11 is reached. The lowest tile label value in B' is $2l_1 + 1$ as all tiles with labels smaller than $2l_1 + 1$ have already been placed into their correct cells. If n' is even, every tile with label $a \in \{2l_1 + 1, 2l_1 + 2, \dots, 2l_1 + n' - 1\}$ is colored black, and if n' is odd, every tile with label $a \in \{2l_1 + 1, 2l_1 + 2, \dots, 2l_1 + n'\}$ is colored black. Thus,

the number of tiles assigned the color black in $\mathcal{A}$ is always odd. By using the inverted version of partition in line 22, the empty space is placed among the black tiles. This makes it so that the subboard bound by l_1 and r contains only black tiles and the empty space. Each tile assigned the color black belongs into this subboard.

We discuss the number of moves `DIVIDEANDCONQUER` requires to solve the puzzle in the worst case.

The number of moves required by `solveRow` in line 8 is analyzed first. Line 11 can only be executed when there are at least 3 numbers in L ; for such an L , we can make the following observation.

Observation 3.1. Let l_1, l_2 and l_3 be the first three numbers in L and let B be the current board state. The tiles that occupy a row from l_1 to l_2 in B^T fall into one of two categories:

- (1) For every tile a occupying one of the cells $2l_1, 2l_1 + 1, \dots, 2l_2$ in B^T , tile a is in one of the cells $2l_1, 2l_1 + 1, \dots, 2l_2, 2l_2 + 1$ in B .
- (2) The tile b occupying the cell $2l_2 + 1$ in B^T is in one of the rows $l_2 + 1, l_2 + 2, \dots, l_3$ in B .

When `solveRow` is executed in line 8, it holds that $l_2 - l_1 \leq 2$. In the case that `solveRow` is called on a row r with $l_1 \leq r < l_2$, every tile occupying r in B^T falls into category (1) and `solveRow` requires a constant number of moves.

We refer to the case where `solveRow` is called on row $r = l_2$ as the *critical* case. When a critical case arises, the tile b defined in (2) is not a constant number of rows away from its target cell; it can be in any row from $(l_2 + 1)$ to l_3 . b is at most $(l_3 - l_2)$ rows away from the target cell in row l_2 . Therefore, `solveRow` requires $O(l_3 - l_2)$ moves in the critical case.

How many moves are required for `solveRow` in total? Naturally, there are at most $n - 1$ cases where it takes a constant number of moves. It remains to analyze the critical cases. There is one occurrence of a critical case for every time the else-case in line 11 is reached. Let B' and n' as well as l_1, l_2 and r be defined as in the else-case of the algorithm. In the else-case, two new subboards are created. `DIVIDEANDCONQUER` solves the upper subboard first and at the end, `solveRow` is called on row r . This is a critical case leading to $O(n'/2)$ moves being made. The upper and the lower subboard have about $n'/2$ rows respectively, and solving them can cause critical cases, too. This is visualized in [Figure 3.9](#).

With the above argumentation, the total number of moves required by `solveRow` in line 8 can be bounded from above by the following sum.

$$1 \cdot \frac{n}{2} + 2 \cdot \frac{n}{4} + \dots + \frac{n}{4} \cdot 2 = \log_2(n) \cdot \frac{n}{2} = O(n \log n)$$

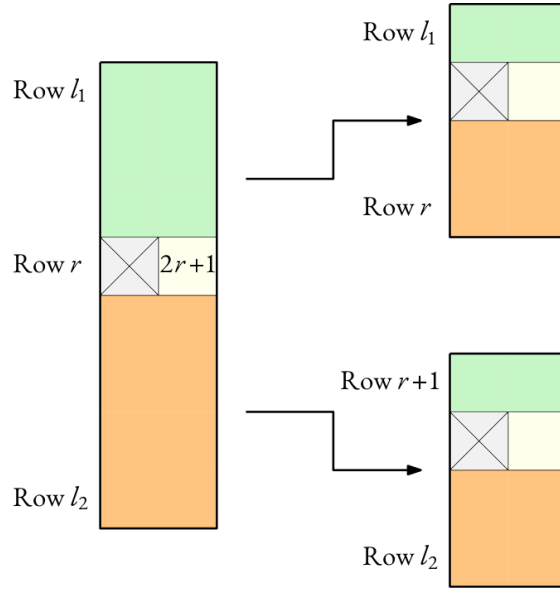


Figure 3.9: An illustration of how critical cases occur in `DIVIDEANDCONQUER`. On the left, the execution of the else-case in line 11 created two subboards indicated by the green and orange colors. `solveRow` was called on every green row already and is now called on row r . This is a critical case as the tile that belongs into column 1 in row r can be in any orange colored cell. There are additional critical cases in the individual subboards as can be seen on the right.

The last term of the sum $((n/4) \cdot 2)$ appears due to each subboard consisting of at least 2 rows.

Next, the number of moves spent on partitioning in step 4 in the worst case must be examined. For a board state B' with n' rows, partition requires $O((n')^2)$ moves and splits B' into two subboards of size roughly $n'/2$. This argumentation leads to the following sum describing the total number of moves `DIVIDEANDCONQUER` spends on the else-case in line 11. We extend the sum to ∞ to obtain an upper bound.

$$\sum_{i=0}^{\infty} 2^i \left(\frac{n}{2^i} \right)^2 = \sum_{i=0}^{\infty} 2^i \frac{n^2}{2^{2i}} = \sum_{i=0}^{\infty} \frac{n^2}{2^i} = O(n^2)$$

Both the if-case and the else-case combined, the `DIVIDEANDCONQUER`-algorithm requires $O(n^2)$ moves to solve the puzzle in the worst case.

3.5.4 2-COLOR VARIATION: PARTITIONONCE

The 2-color variation is ideal for PARTITIONONCE as the tile labels directly specify the assignment to colors. The `partition`-procedure needs only a simple extension and we obtain the PARTITIONONCE-algorithm that solves the 2-color puzzle.

Algorithm 11: PARTITIONONCE (B)

Input: A board state B in the 2-color variation.

- 1 Let A be an assignment of the tiles in B .
 - 2 **foreach** *tile* in B **do**
 - 3 **if** the tile has a label $a = 1$ **then**
 - 4 Assign the tile the color black in A .
 - 5 **else**
 - 6 Assign the tile the color white in A .
 - 7 Use `partition` on B according to A .
 - 8 Move the empty space to cell $2n - 1$.
-

As shown, one call of `partition` requires $O(n^2)$ moves. This bound applies to the PARTITIONONCE-algorithm as well.

3.5.5 k -COLOR VARIATION: COLORBYCOLOR

The COLORBYCOLOR-algorithm solves a given k -color puzzle from top to bottom. This is done by iterating over the labels $1, 2, \dots, k$ in that order; in iteration i , COLORBYCOLOR puts all tiles labeled i into their correct cells. Specifically, for a board state B , let q be the number of tiles that have a label $a \leq i$. After iteration i , we have that $c_j = c_j^T$ for all $j \in \{0, 1, \dots, q - 1\}$.

Each iteration involves one call of `partition`. In iteration i , the COLORBYCOLOR-algorithm separates the tiles with label i from the tiles with label $a > i$. The algorithm does this without moving tiles with label $b < i$, as these tiles are already placed in their correct cells. Therefore, `partition` is only used on the part of the board that is not solved yet.

We now formally describe the algorithm.

Algorithm 12: COLORBYCOLOR (B)

Input: A board state B in the k -color variation.

```

1 Initialize  $t \leftarrow 0$ .
2 for  $i \leftarrow 1$  to  $k - 1$  do
3   Let row  $b$  be the lowest row that contains a tile with label  $i$ .
4   if the empty space is in a row below row  $b$  then
5     Move the empty space up to row  $b$ .
6     if row  $b + 1$  now contains a tile with label  $i$  then
7       Set  $b \leftarrow b + 1$ .
8   Let  $B'$  be the subboard of  $B$  bound by  $t$  and  $b$ .
9   Let  $q$  be the amount of tiles with label  $i$  in  $B'$ .
10  Let  $A$  be an assignment of the tiles in  $B'$ .
11  foreach tile in  $B'$  do
12    if the tile has a label  $a = i$  then
13      Assign the tile the color black in  $A$ .
14    else
15      Assign the tile the color white in  $A$ .
16  Use partition on  $B'$  according to  $A$ .
17  Set  $t \leftarrow t + \lfloor q/2 \rfloor$ .
18  if  $q$  is odd then
19    Use solveRow on row  $t$ .
20    Set  $t \leftarrow t + 1$ .
21 Move the empty space to cell  $2n - 1$ .
```

The variable t denotes the number of rows that have already been solved by COLORBYCOLOR. In line 5, moving the empty space to row b could move a tile with label i down one row, and b is increased in that case. Every tile with label i is assigned the color black in A and therefore moved into a cell it belongs to. In row 18, if q is odd, row t is only partially solved as partition placed an arbitrary tile with label $a > i$ into cell $2t+1$. Solving row t is therefore achieved with solveRow.

There are certain cases that the algorithm needs to handle separately. If for any i there is only one tile with label i , using partition in line 16 is not necessary in iteration i . The behavior of COLORBYCOLOR also needs to be adjusted if there are multiple iterations left when $t \in \{n - 2, n - 1\}$ is one of the two bottom rows. In that case, rows $n - 2$ and $n - 1$ are solved using OPT.

The case mentioned in the analysis of RowByRow (Algorithm 8), where the puzzle cannot be solved after a pair of tiles with the same label has been placed in the wrong order, could also

occur in COLORBYCOLOR. This case is handled similarly here: Again, the highest numbered pair of cells $i, i + 1$ is determined such that i and $i + 1$ contain tiles with the same label a in B^T . Then, COLORBYCOLOR only uses `partition` to solve the rows that are above the row r containing cell i . Once row r is reached, the remaining board is solved in the same manner as in ROWBYROW.

In COLORBYCOLOR, `partition` is called at most $k - 1$ times. As a generous upper bound, we can state that each call of `partition` requires $O(n^2)$ moves. Therefore, $O(k \cdot n^2)$ moves are required in total for the COLORBYCOLOR-algorithm in the worst case.

This concludes the discussion of the various puzzle-solving algorithms. In the following chapter, these algorithms will be evaluated and compared.

4 EMPIRICAL ANALYSIS

The algorithms proposed in the last chapter all have very similar behavior when it comes to the asymptotical number of moves they require in the worst case. From our theoretical analysis, we saw an upper bound of $O(n^2)$ for every algorithm except the `colorByColor`-algorithm in the k -color variation, where the bound is $O(k \cdot n^2)$.

In this chapter, an empirical analysis of the algorithms is conducted. This allows for a more realistic and practical evaluation of the performance of the algorithms.

4.1 TEST RESULTS AND EVALUATION

The algorithms were implemented in C++ [10]. The project including the algorithms and the tests can be found on the accompanying CD (Appendix A) as well as on Github at <https://github.com/Vikplusplus/nx2-puzzle-code>. The project includes a web application allowing users to interactively play the various $n \times 2$ -puzzle variations and provides functionality to visualize the solution move sequences produced by each algorithm.

The experiments were conducted as follows: For each variation, every algorithm is tested on a large number of randomly generated board states. For different numbers of rows n , we plot the average number of moves required by each algorithm to solve board states of that size. The plots are generated using the matplotlib-library [12] in Python [13].

4.1.1 NUMBERED VARIATION

For every $n \in \{5, 6, \dots, 300\}$, 1000 solvable board states were generated by randomly shuffling the target state array $[1, 2, \dots, 2n - 1, 0]$ until the resulting board state was solvable. For each board state, all three algorithms are applied to solve it, and the number of moves required by each algorithm is recorded. For every n , taking the average of that number over all 1000 tests results in the plot seen in Figure 4.1.

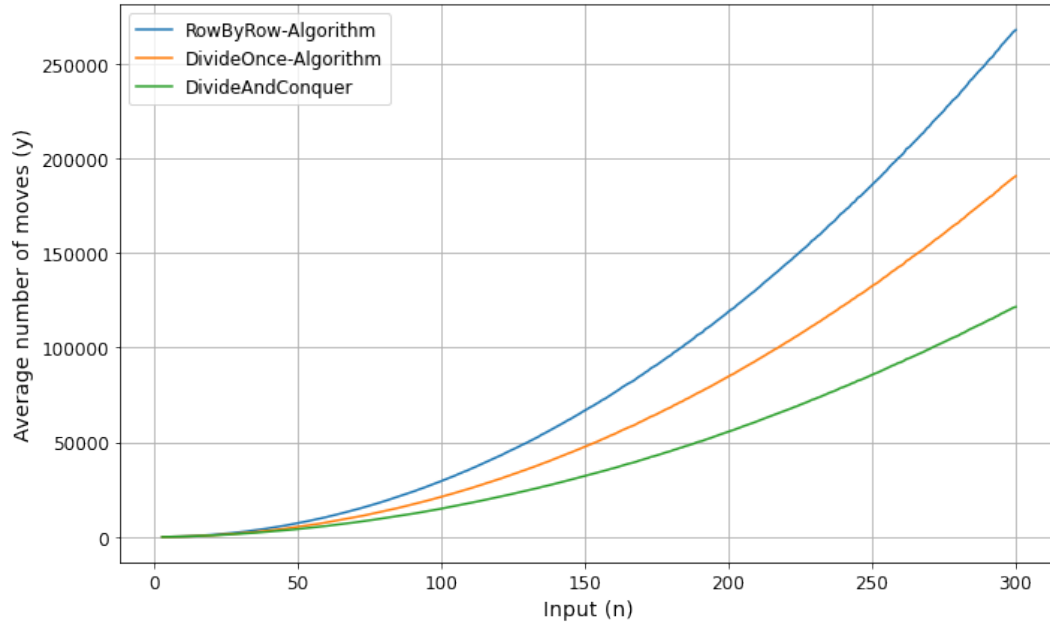


Figure 4.1: Empirical evaluation results of the numbered variation algorithms.

We can generally observe that the number of moves required for boards of size n grows superlinearly in n regardless of the algorithm used. Still, there are noticeable differences between the algorithms. The `DIVIDEANDCONQUER`-algorithm, where the board is partitioned multiple times, is the most efficient. It consistently required less than 50% of the number of moves that the `ROWBYROW`-algorithm needed. For $n = 300$, `DIVIDEANDCONQUER` did 121,566 moves on average against 267,862 moves done by `ROWBYROW`. `DIVIDEONCE` uses partition once before `ROWBYROW` is applied; this already leads to an increased efficiency of more than 25%. We can presume that using `partition` is efficient as it creates parts of the board that `ROWBYROW` can solve mostly independent from one another: One call of `partition` produces two parts of *half* the previous board size and with the superlinearly growing curve of `ROWBYROW`, the moves needed to solve each part is cut by a factor *greater than 2* compared to before.

4.1.2 2-COLOR VARIATION

Similarly to the numbered variation, 1000 random board states were generated for each $n \in \{5, 6, \dots, 300\}$. Each board state had a random number of black tiles of $1 + z$ where z follows a binomial distribution $B(2n - 3, 0.5)$. After z was selected, we can compute the target state array for a 2-color board state having $1 + z$ tiles with label 1. The random board state is obtained by randomly shuffling that target state. Figure 4.2 shows the resulting data.

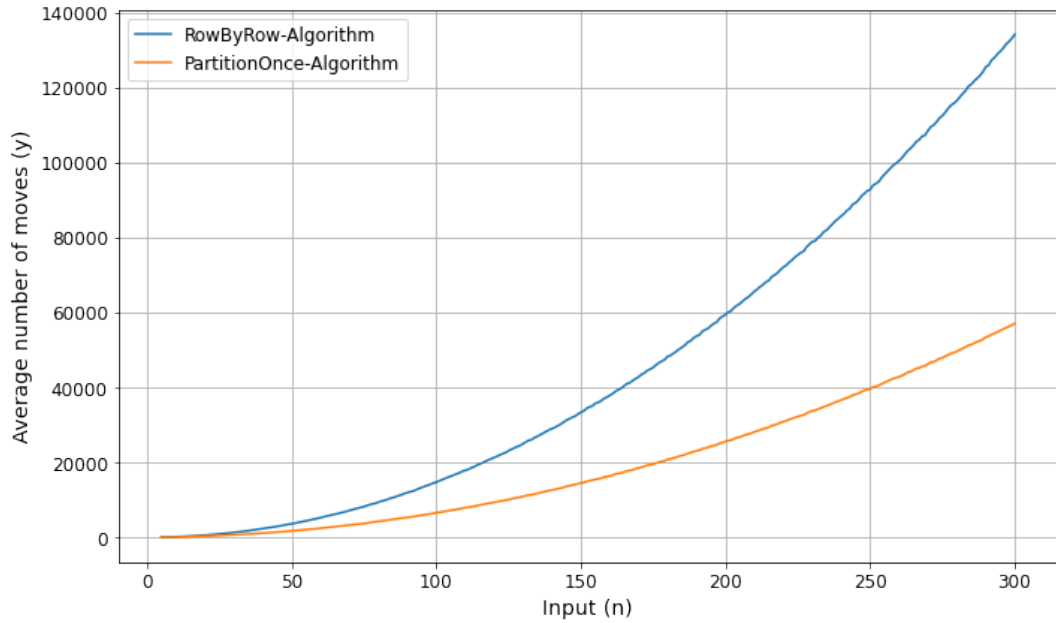


Figure 4.2: Empirical evaluation results of the 2-color variation algorithms.

We can see that `PARTITIONONCE` is the algorithm of choice for the 2-color variation. It generally needs less than half the number of moves of `ROWBYROW`, which is a difference of around 80,000 moves for $n = 300$. Comparing this plot to the one from the numbered variation, it can be observed that the puzzles of this variation are solved in fewer moves on average. The `ROWBYROW`-algorithm performs about twice as well as in the numbered variation. This is only natural since in the 2-color variation, `ROWBYROW` only uses `solveRow` on rows that contain a black tile in the target state. This concerns the first half of the rows on average, since z has an expected value of $n - 1.5$. Comparing the best-performing algorithms of the variations (`PARTITIONONCE` and `DIVIDEANDCONQUER`), we see that they also differ by a factor of about 2 in their performance.

4.1.3 k -COLOR VARIATION

We test the algorithms in the k -color variation in two scenarios. In the first, $k = 5$ is fixed and tests are conducted for each $n \in \{5, 6, \dots, 300\}$. In the second, $n = 300$ is fixed and the input is $k \in \{5, \dots, 300\}$. That way, the individual effect that each of the parameters has on the difficulty of the puzzle can be observed. As before, 1000 random board states are generated for each n and for each k . The first plot, where $k = 5$ is fixed, can be seen in [Figure 4.3](#).

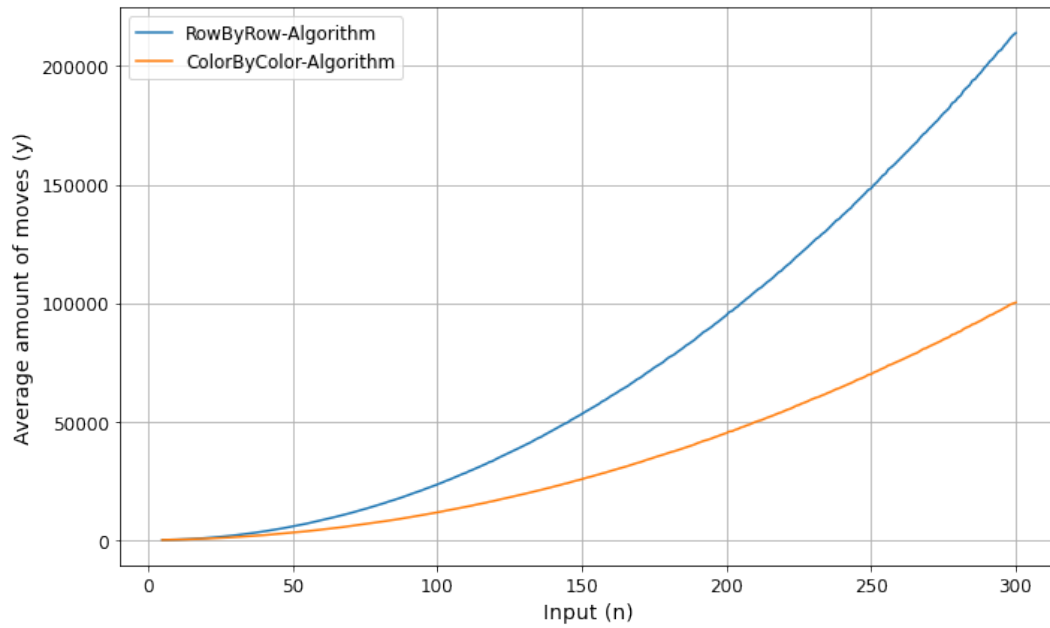


Figure 4.3: Empirical evaluation results of the k -color variation algorithms with fixed $k = 5$.

This plot shows an image comparable to the 2-color variation. `COLORBYCOLOR` is the better algorithm for $k = 5$. It consistently solves the puzzles in less than half the number of moves compared to `ROWBYROW`. Raising the number of possible label values from 2 in the 2-color variation to 5 here led to a noticeable increase in the number of moves `ROWBYROW` requires on average.

For the second plot of the k -color variation, we fix $n = 300$. The results are illustrated in Figure 4.4.

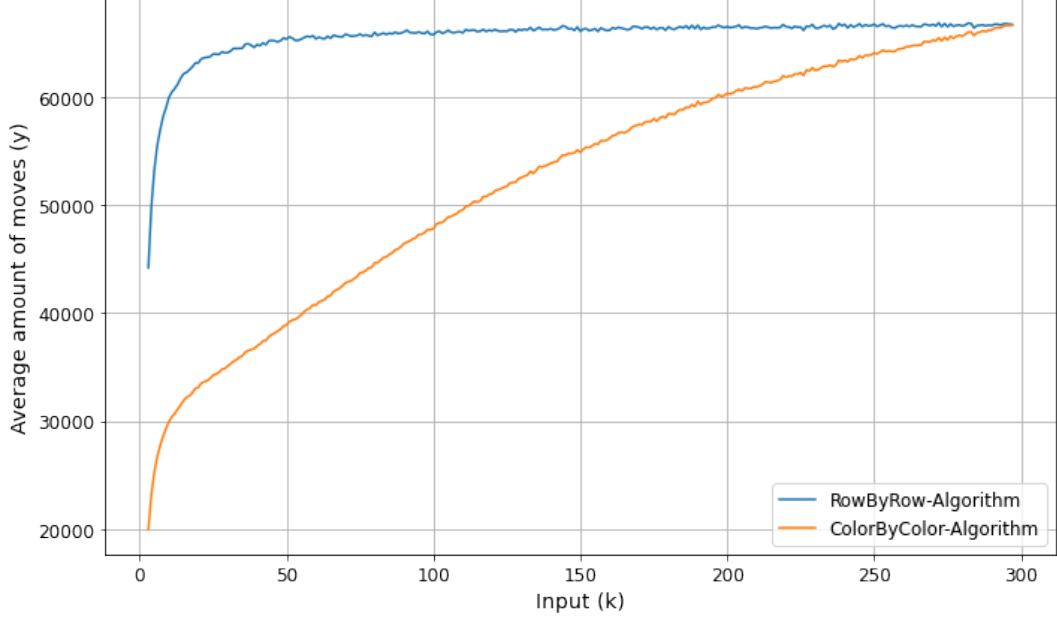


Figure 4.4: Empirical evaluation results of the k -color variation algorithms with fixed $n = 300$.

We can generally observe that on average, both algorithms require more moves to solve the puzzle as k increases. However, this effect decreases as k gets larger. This is especially noticeable for the RowByRow-algorithm; for large k of more than 100, its performance gets only marginally worse. While the difference between $k = 3$ and $k = 20$ is still around 20,000 moves, the increase from that point until $k = 298$ is only about 3000 moves. In the k -color variation, the RowByRow-algorithm only uses `solveRow` until rows with label k in the target state are reached. Meanwhile, solving a row requires more moves if the row is further towards the top of the board. This could explain why the curve of RowByRow is steeper for smaller k in this experiment.

A similar trend can be seen for the COLORBYCOLOR-algorithm, but the rate at which the number of moves increases is consistently higher than in RowByRow. This leads to the difference in performance between the two algorithms becoming smaller as k increases until both algorithms are equally performant at $k = 298$. This can be explained by the fact that COLORBYCOLOR relies more significantly on `solveRow` as k increases.

4.1.4 GENERAL OBSERVATIONS

Across all variations, there are 2 points that stand out.

The first is that the more an algorithm uses `partition`, the more efficient it becomes overall. In the numbered variation, `DIVIDEANDCONQUER` uses `partition` the most and it is the best algorithm of the three that were proposed. In the 2- and the k -color variations, the `PARTITIONONCE`- and `COLORBYCOLOR`-algorithms both use `partition` mainly and they outperformed `ROWBYROW` by a large margin in the experiments. The data suggests that the extra moves afforded, to use `partition`, are outweighed by the effect that each tile is sorted into the correct part of the board.

The second observation that can be made is that between the variations, the puzzles become more difficult to solve when there is a larger number of possible tile label values. This was recognizable in the `ROWBYROW`-algorithm which got noticeably less performant when going from the 2-color variation to the k -color variation with $k = 5$. This was also observable clearly in the experiment where n was fixed in the k -color variation. This observation can be explained by the increase in combinatorial complexity that occurs when the number of possible tile label values goes up. We analyzed this in detail when discussing the search tree size of `OPT` in [Section 3.2](#). Intuitively, a board state where many tiles have the same label is easier to solve, since a given cell can be solved with multiple tiles.

5 CONCLUSION

In this work, we investigated $n \times 2$ -puzzles in three variations. We discussed the solvability of each variation and found that while the 2-color and k -color variations can always be solved, the numbered variation is only solvable for half of all board states. In chapter 3, we discussed the complexity of $n \times 2$ -puzzles and saw that in each variation, there are board states where $\Omega(n^2)$ moves are needed to solve the puzzle. We then proposed the `solveRow`- and `partition`-procedures and used them to design puzzle-solving algorithms for each variation. In chapter 4, we carried out an empirical analysis of these algorithms.

A key interest of this thesis was to see whether variations with a lower number of possible tile labels, such as 2-color and k -color, were less difficult than the numbered variation. We found no difference in asymptotic complexity in the theoretical analysis, but our experiments from chapter 4 show that on average, 2-color board states can be solved with less moves than numbered board states. However across all variations, the number of moves made by the algorithms grew superlinearly in n .

Another important result of the thesis is that algorithms based on the `partition`-routine generally outperform those using `solveRow`. In the numbered and 2-color variations, we observed that the best algorithms used less than half the number of moves compared to the `RowByRow`-algorithm which relies only on `solveRow`.

The algorithms proposed in this work can be further improved upon. Possible improvements are:

- `solveRow` moving both tiles simultaneously to the given row instead of lifting them up one by one. This could be done by first moving the two selected tiles to adjacent cells before starting to move them to the given row.
- In the `DIVIDEANDCONQUER`-algorithm, we currently use `partition` until the subboards have less than four rows. Altering this threshold could lead to better performance.
- A new k -color algorithm that functions similarly to the `DIVIDEANDCONQUER`-algorithm in the numbered variation. Judging from the efficiency of `DIVIDEANDCONQUER` in the numbered variation, this algorithm could outperform the two k -color algorithms proposed in this work.

Doing the same experiments on the improved algorithms could lead to different results.

Although this was not the focus of this work, future research could aim to develop parallelized versions of the `DIVIDEONCE`- and `DIVIDEANDCONQUER`-algorithms to reduce their running time. A problem that would have to be solved is that adjacent subboards are not entirely independent from one another. Future work could also include a generalization of the algorithms proposed here to puzzles of size $n \times m$ or $n \times n$. Similarly, 3-dimensional puzzle boards of size $n \times 2 \times 2$ could be studied.

An important open question is whether finding an optimal solution (i.e. with the least number of moves) for any of the $n \times 2$ -puzzle variations is $\mathcal{NP}$ -hard. $\mathcal{NP}$ -hardness could be shown by reduction from a known $\mathcal{NP}$ -hard problem; the numbered puzzle with board size $n \times n$ is known to be $\mathcal{NP}$ -hard [5] and might be an appropriate starting point for a reduction. Conversely, future research could find a polynomial time optimal algorithm to show that a variation is in $\mathcal{P}$. The existence of such an algorithm is presumably most likely in the 2-color variation as it has the least combinatorial complexity.

BIBLIOGRAPHY

- [1] W. W. Johnson and W. E. Story, “Notes on the “15” puzzle.,” *American Journal of Mathematics*, 1879.
- [2] S. Loyd, *Sam Loyd’s Cyclopedia of 5000 Puzzles, Tricks, and Conundrums With Answers*. 1914.
- [3] A. Brünnger, A. Marzetta, K. Fukuda, and J. Nievergelt, “The parallel search bench zram and its applications,” 1999.
- [4] R. E. Korf and P. Schultze, “Large-scale parallel breadth-first search,” 2005.
- [5] D. Ratner and M. Warmuth, “Finding a shortest solution for the $n \times n$ extension of the 15-puzzle is intractable,” 1986.
- [6] K. Takano, “Self-supervision is all you need for solving rubik’s cube,” 2023.
- [7] O. Goldreich, “Finding the shortest move-sequence in the graph-generalized 15-puzzle is np-hard,” 1984.
- [8] M. Beyer, S. Mereta, É. Roldán, and P. Voran, “Higher-dimensional cubical sliding puzzles,” 2023.
- [9] Z. Zhong, “Additive approximation algorithm for sliding puzzle,” 2023.
- [10] cppreference.com, “C++ reference.” <https://en.cppreference.com/>, 2024. Accessed July 2025.
- [11] A. F. Archer, “A modern treatment of the 15 puzzle,” *The American Mathematical Monthly*, 1999.
- [12] J. D. Hunter, “Matplotlib: A 2d graphics environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
- [13] Python Software Foundation, *Python 3*. Python Software Foundation, 2024. <https://docs.python.org/3/>.
- [14] OpenAI, “Chatgpt (july 2025 version).” <https://chat.openai.com>, 2025.

LIST OF FIGURES

1.1	Basic 15-puzzle example	2
1.2	Sam Loyd's unsolvable 15-puzzle	3
2.1	$n \times 2$ -puzzle board layout	5
2.2	Examples for cell notation in each variation	6
2.3	Applying a move to a board state	7
2.4	Applying a move sequence to a board state	8
2.5	Example target board states for each variation	9
2.6	Snake-like cell numbering	10
2.7	Example for slot notation	11
2.8	Equivalency between board states with the same slot notation	12
2.9	Applying a move permutation	15
2.10	Equivalent transpositions in cell and slot notation	19
2.11	Example marked board state	22
2.12	Marked board state representation	22
3.1	Example search tree for <code>OPT</code>	27
3.2	Example move sequence in <code>solveRow</code>	31
3.3	Example move sequence in <code>liftTile</code>	32
3.4	Examples for resulting board states after applying each phase of <code>partition</code>	33
3.5	Example for the cyclic shift by <code>partition</code> in phase 1	35
3.6	Example for the cyclic shift by <code>partition</code> in phase 2	37
3.7	Example for the cyclic shift by <code>partition</code> in phase 3	38
3.8	Visualization of the approach of <code>DIVIDEANDCONQUER</code>	41
3.9	Visualization of the occurrence of critical cases in <code>DIVIDEANDCONQUER</code>	44
4.1	Empirical evaluation results of the numbered variation algorithms	49
4.2	Empirical evaluation results of the 2-color variation algorithms	50
4.3	Empirical evaluation results of the k -color variation algorithms with fixed k	51
4.4	Empirical evaluation results of the k -color variation algorithms with fixed n	52

LIST OF ALGORITHMS

1	BOARDSOLVABLE(B)	20
2	OPT(B)	29
3	solveRow(B, r)	30
4	liftTile(B, r, y)	31
5	partition-phase1(B, A)	34
6	partition-phase2(B, A)	36
7	partition-phase3(B)	37
8	RowByRow(B)	39
9	DIVIDEONCE(B)	40
10	DIVIDEANDCONQUER(B)	42
11	PARTITIONONCE(B)	45
12	COLORBYCOLOR(B)	46

A SUPPLEMENTARY MATERIAL

The attached CD contains the following material:

- `Algorithms/` – Source code for the puzzle-solving algorithms.
- `WebAPI/` – Source code for the web application.
- `main.cpp` – Entry point for the project execution.
- `CMakeLists.txt` – CMake configuration.
- `LICENSE` – License and usage information.
- `README.md` – Instructions for running the code.

B TOOLS AND AIDS

The following table lists key tools and resources utilized throughout the course of this thesis.

Ref.	Tool	Form of use
[14]	ChatGPT-4 (OpenAI)	assistance with the grammar and formulation, assistance with debugging the code
[10]	C++	implementation and testing of the algorithms, implementation of the web application
[13]	Python 3 (Python Software Foundation)	generation of the plots
[12]	Matplotlib (Hunter, J. D.)	generation of the plots

Bitte dieses Formular zusammen mit der Abschlussarbeit abgeben!

Erklärung zur Abschlussarbeit

gemäß § 35, Abs. 16 der Ordnung für den Bachelorstudiengang Informatik
vom 17. Juni 2019:

Hiermit erkläre ich

Rieß, Viktor

(Nachname, Vorname)

Die vorliegende Arbeit habe ich selbstständig und ohne Benutzung anderer als der angegebenen Quellen und Hilfsmittel verfasst.

Ich bestätige außerdem, dass die vorliegende Arbeit nicht, auch nicht auszugsweise, für eine andere Prüfung oder Studienleistung verwendet wurde.

Zudem versichere ich, dass alle eingereichten schriftlichen gebundenen Versionen meiner vorliegenden Bachelorarbeit mit der digital eingereichten elektronischen Version meiner Bachelorarbeit übereinstimmen.

Frankfurt am Main, den



Unterschrift der/des Studierenden